

1일차 강의 자료 · 8시간

Cursor AI를 활용한 소프트웨어 개발

Prompt Engineering · Concept-to-Code Traceability · Dual-Track TDD · PRD

1교시	Prompt Engineering & Cursor AI 설치	P-C-T-F · 7단계 · ICL/CoT/ReAct
2·3교시	아름다운 SW — 해석학적 접근	C2C Traceability · 의사결정 파이프라인
4교시	Magic Square × C2C × 문제인식	온톨로지 · ECB 분석 · 문제인식 실습
5교시	Dual-Track TDD 방법론	왜 늦게 실패? · Problem · Scenarios
6교시	Magic Square — Dual-Track TDD 요구사항	UX Contract · Logic Rule 작성
7교시	Toy Project PRD + Cursor AI 개발 시작	PRD 작성 · AI Assist 코딩 시작
8교시	To-Do 구조 작성 원리	체크박스 · 속성 테스트 · 검증 포인트

1일차 시간표 (8시간)

일정

1교시 60분 Prompt Engineering & Cursor AI P-C-T-F · 7단계 · Few-Shot · CoT · ReAct · Cursor 설치 및 Git

2교시 60분 아름다운 소프트웨어 — 해석학적 접근 C2C Traceability · 의사결정 파이프라인 · 문제인식 + 온톨로지 + 프롬프팅 체인

3교시 60분 (2교시 연속) ECB 모델 · 4계층 구조 · Semantic Erosion 패턴

4교시 60분 Magic Square × C2C × 문제인식 Magic Square 소개 · ECB 분석 실습 · 문제인식 작성

// 점심 휴식 60분

5교시 60분 Dual-Track UI+Logic TDD + Toy Project 소개 왜 늦게 실패? · Problem & Scenarios · Toy Project 소개

6교시 60분 Magic Square — Dual-Track 요구사항 작성 UX Contract · Logic Rule · 3단 매핑 표

7교시 60분 Toy Project PRD 작성 + Cursor AI로 개발 시작 Gherkin · PRD 구조 · Cursor Agent 활용

8교시 60분 To-Do 구조 작성 체크박스·번호체계·Requirements 추적·속성 테스트·검증 포인트

01

Prompt Engineering & Cursor AI

고급 프롬프트 설계 기본기 · P-C-T-F · 7단계 · ICL/CoT/ReAct · Cursor 설치

1교시

60분

생성형 AI란 무엇인가?

AI Literacy = AI 이해 + AI 활용 + AI 판단 능력 — 2026년은 코딩보다 "의도를 정확히 전달하는" 역량이 핵심

생성형 AI (Generative AI)

프롬프트에 대응해 텍스트·이미지·코드 등 콘텐츠를 생성하는 AI 시스템
입력 데이터의 패턴·구조를 학습 → 유사 특성의 새 데이터 생성
대표 모델: ChatGPT, Claude, Gemini, Midjourney, DALL-E

LLM (Large Language Model)

Transformer 기반 딥러닝 모델
방대한 텍스트 데이터로 사전 학습(Pre-training)
자연어 이해·생성·번역·코딩 등 다목적 활용
핵심: Attention 메커니즘 → 문맥 이해

Prompt → Model → Output 핵심 흐름

① Prompt

사용자 입력
자연어 지시



② Tokenize

텍스트 → Token
숫자 벡터 변환



③ Attention

문맥 이해
중요 단어 집중



④ Generate

다음 토큰 예측
답변 완성

AI와 기계학습의 발전 과정 (트랜스포머 이전)



핵심 문제: RNN/LSTM은 문장이 길어질수록 초반 정보를 "압축-손실"함 → Thought Vector 병목 → Attention으로 해결

트랜스포머 아키텍처의 혁명

Attention 메커니즘

- Encoder가 입력 전체를 하나의 벡터로 압축하는 병목을 제거
- Decoder가 매 스텝마다 입력 전체를 참조 → 필요한 부분에 집중

"Are you free tomorrow?" → 번역 시 각 단어가 어디에 주목하는지 계산

나는 지금 '시간'까지 말했고, 다음 단어를 만들려면 입력 문장에서 뭘 봐야 할까?"라는 질문 벡터時

- $Q(\text{Query}) \cdot K(\text{Key}) \cdot V(\text{Value})$ 의 내적으로 중요도 계산
- 결과: Are: 0.00 you: 0.01 free: 0.74 tomorrow: 0.24 ?: 0.01
→ context 벡터는 74%는 "free"의 의미 + 24%는 "tomorrow"의 의미를 담고 있음

Transformer가 해결한 것

고전 Seq2Seq	Transformer
고정 벡터 병목	병목 없음
장거리 의존성 X	어디서나 참조 
순차 처리(느림)	병렬 처리(빠름)
LSTM 기반	Self-Attention

2017년 이후 GPT·BERT·LLaMA·Claude 등 모든 SOTA 모델은 Transformer 기반

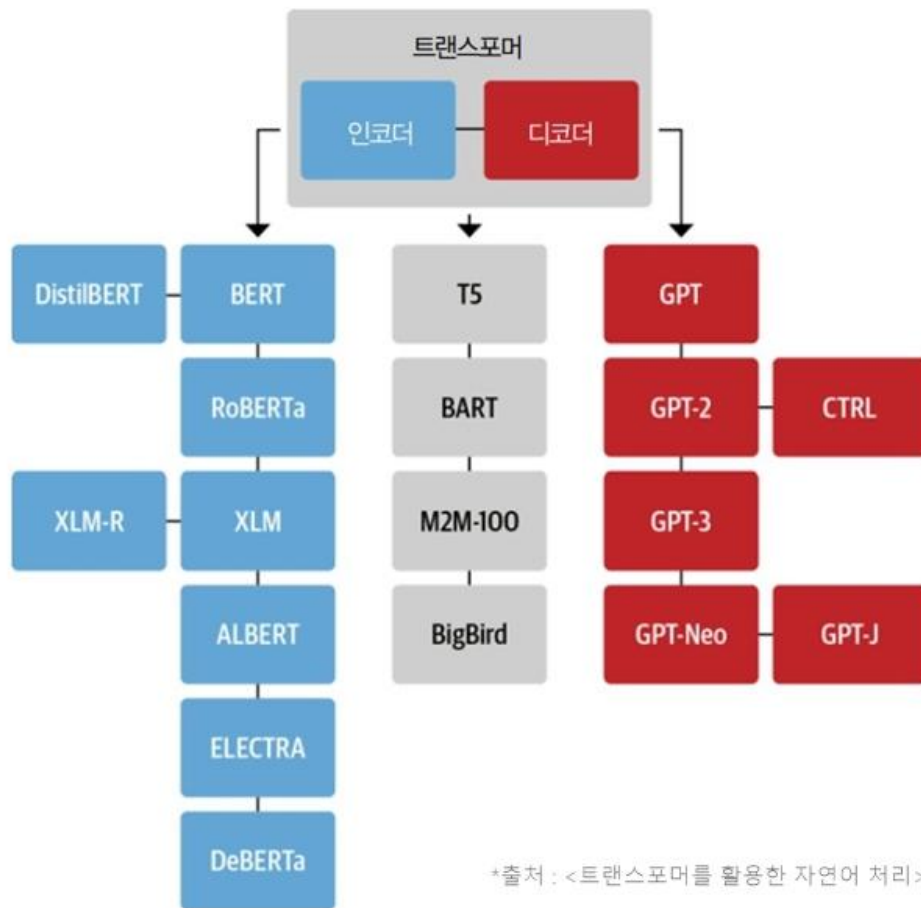
BERT 와 GPT



전략적 감독과 판단



책임



*출처 : <트랜스포머를 활용한 자연어 처리>, 한빛미디어, 2022

Transformer · Attention · RLHF — LLM 작동 원리

⚡ Attention 메커니즘 — LLM의 핵심

Query

현재 단어가 "무엇을 보아야 하나?" 라는 질문 벡터

Key

입력 문장 각 단어의 "식별 태그"

Value

실제 의미 정보 — 가중 평균으로 Context 생성

Softmax

유사도 점수를 확률 분포로 정규화

Context

"free 74% + tomorrow 24%" 처럼 중요 단어 집중

→ 2017년 이후 모든 SOTA(GPT·Claude·Llama)는 Attention 기반 Transformer 사용

🎯 RLHF — ChatGPT를 만든 기술

①

사전 학습

GPT-3 등 대규모 텍스트
비지도 학습으로 언어 능력 습득

②

초기 응답 생성

프롬프트 → 여러 응답 생성
인간 평가자가 품질 평가

③

보상 모델 학습

평가 데이터로 Reward Model 훈련
"좋은 응답"을 예측하는 모델

④

강화 학습 적용

PPO 알고리즘으로 LLM 파인튜닝
높은 보상을 받는 응답 강화

Closed vs Open Source LLM

Closed: GPT-4o, Claude 3.5 → API 방식, 높은 성능
Open : Llama 3.1, Qwen 2.5 → 로컬 실행, 보안/비용
↑

Prompt → Model → Output 흐름



SOTA는 State-of-the-Art의 약자로, AI/ML 분야에서 특정 작업(예: 텍스트 생성, 이미지 인식, 번역 등)에 대해 현재 최고 성능을 내는 모델

분야	SOTA 모델 예시	주요 강점	벤치마크 예시 (2025년 최고 점수)
LLM (NLP)	GPT-4o (OpenAI)	실시간 대화, 멀티모달 (텍스트+이미지+음성), 함수 호출 지원	MMLU: 88%+, HumanEval: 90%+
LLM (오픈소스)	Llama 3.1 (Meta), Qwen 2.5 (Alibaba)	오픈소스라 fine-tuning 쉽고, 컨텍스트 윈도우 128K+ 토큰 지원	MT-Bench: 8.5/10, GSM8K: 95%
멀티모달	Gemini 1.5 (Google), DALL-E 3 (OpenAI)	이미지/비디오 생성+이해, MMMU 같은 시험에서 인간 수준	MMMU: 65%+, VQA: 85%
비전 (CV)	DINOv3 (Meta), Segment Anything (SAM, Meta)	자가 지도 학습으로 이미지 세그멘테이션, 1B+ 마스크 데이터셋	ImageNet: 90%+, COCO: 60% mAP
추론/에이전트	o3-mini (OpenAI), Claude 3.5 (Anthropic)	구조화된 출력, 도구 통합 (tool calling), 비용 효율적	ARC: 75%+, GPQA: 50%+
기타 (RAG/에이전트)	RAG-Anything, Agent0	멀티모달 지식 검색, 자가 진화 에이전트 (인간 데이터 없이 학습)	복합 벤치마크: 20%+ 향상

프롬프트 엔지니어링

프롬프트(Prompt)란?

인공지능에 지시하는 명령어 또는 AI와의 대화를 위한 질문, 지시, 요청.

더 넓게는 AI가 의도를 이해하고 적절한 응답을 생성하도록 안내하는 모든 형태의 입력.

마치 음악가가 악기를 다루듯,
작가가 펜을 다루듯,
프롬프트 엔지니어는 언어를 통해 AI의 잠재력을 이끌어냄.

좋은 프롬프트의 효과

왜 중요한가?

같은 AI라도 프롬프트를 어떻게 작성하느냐에 따라 전혀 다른 결과 제공.

나쁜 예:

'코드 짜줘'

→ AI: '어떤 코드를 원하시나요?'

좋은 예 (P-C-T-F):

'너는 Python 전문가 [Persona].

사용자 인증 시스템 개발 중 [Context].

JWT 기반 로그인 함수 작성 [Task].

타입 힌트 포함, 주석 한국어 [Format].'

→ 정확한 코드 즉시 생성

프롬프트란? — AI와 대화하는 언어

나쁜 프롬프트 vs 좋은 프롬프트 비교

✖ 나쁜 프롬프트

프롬프트

홈페이지 만들어줘

문제점:

- ▶ 목적·대상·스타일 정보 없음
- ▶ AI가 방향성 없이 추측해서 생성
- ▶ 결과: 일반적인 기본 템플릿
- ▶ 재작업 3~5회 필요

✔ 좋은 프롬프트

프롬프트

오늘 날씨와 주요 뉴스를 보여 주는
나만의 시작 홈페이지를 만들고 싶어.
구글 스타일 디자인으로, 할 일 목록도 포함해 줘.

개선된 점:

- ▶ 목적·스타일·기능이 명확
- ▶ AI가 정확한 방향으로 실행
- ▶ 결과: 맞춤형 홈페이지 완성
- ▶ 1~2회 수정으로 완성

좋은 프롬프트의 6가지 핵심 원칙

Principles of Effective Prompting

1 명확성 (Clarity)

모호함 없이
정확하게 의도 전달

2 구체성 (Specificity)

필요한 세부 사항을
충분히 제공

3 맥락 제공 (Context)

배경 정보로
AI 상황 이해 지원

4 구조화 (Structure)

복잡한 요청은
논리적으로 구성

5 목적 정의 (Purpose)

무엇을, 왜
필요한지 명확히

6 제약 설정 (Constraints)

형식, 길이,
스타일 등 지정

CLEAR 프레임워크

좋은 프롬프트를 만드는 5가지 구성 요소



Context (맥락)

VSCode + C++17 CMake 환경 (Java: Maven/Spring Boot 병행)



Length (길이)

예제 1개만 / 전체 코드 / 단계별 5개로



Examples (예시)

Before/After 코드, 원하는 출력 형태 샘플



Ask (요청)

만들어줘 / 분석해줘 / 수정해줘 (동사로 시작)



Role (역할)

시니어 C++17 개발자로서 / Java 17 전문가 입장에서

💡 CLEAR 적용 예: "시니어 Java 17 개발자로서(R), VSCode Maven 환경에서(C), 할 일 REST API를 만들어줘(A). GET/POST/DELETE 포함, JUnit 5 테스트까지(E), 전체 코드 완성본으로(L)."

역할 프롬프팅 (Role Prompting)

AI에게 전문가 페르소나를 부여해 전문적 결과 유도

Java 백엔드 개발자

Spring Boot · JPA · RESTful API · 마이크로서비스

C++ 시스템 개발자

STL · RAII · 스마트포인터 · 성능 최적화

코드 리뷰어

버그 탐지 · 컨벤션 검토 · 보안 취약점 분석

소프트웨어 아키텍트

설계 패턴 · 계층 분리 · 확장성 · 의존성 관리

QA 엔지니어

JUnit 5 · Google Test · 경계값 · 통합 테스트

기술 문서 작성자

API 문서 · README · 아키텍처 다이어그램 · JavaDoc

💡 실습: "시니어 Java 17 백엔드 개발자로서, 이 TodoService 코드를 리뷰해 줘. NullPointerException 위험·트랜잭션 처리·성능 이슈 중심으로 검토해 줘."

Prompt Engineering : P-C-T-F & 7단계 프레임워크

AI 응답 품질의 80%는 프롬프트 설계에 달렸다 — 같은 AI라도 프롬프트에 따라 전혀 다른 결과

P-C-T-F 패턴 (Anthropic/IBM 기본 패턴)

- P Persona**
역할 부여
"너는 시니어 백엔드 개발자야"
- C Context**
배경 상황
"현재 주문 시스템 설계 중"
- T Task**
수행 과제
"RESTful API 설계안 3가지 제안"
- F Format**
출력 형식
"각각 장단점을 표로 정리"

7단계 프레임워크 (통합 버전)

- 1 Role & Goal** "기획 전문가, 목표: 요구사항 분석"
- 2 Context & Input** "현재 신규 쇼핑몰 프로젝트"
- 3 Constraints** "300자 이내, 안전 규칙 준수"
- 4 Output Format** "마크다운 표 형식으로"
- 5 Example** "예시: 문제→해결 3가지"
- 6 CoT** "Step by step으로 생각해"
- 7 Feedback Loop** "응답 검토 후 재질문"

→ CRISP 기반 통합: Context+Role → Input+Steps(CoT) → Product

프롬프트 기법 비교 정리

상황별 최적 기법 선택

기법	핵심 원리	사용 상황	효과
P-C-T-F	역할·맥락·작업·형식 4단계	일반 코딩, 분석 요청	★★★★☆
7-Step	목표~반복 7단계 통합	복잡한 기획, 설계 작업	★★★★★
KERNEL	간단·설명·정제·좁힘·탐색·학습	빠른 반복 개선	★★★★☆
Few-Shot	예시를 통한 패턴 학습	특정 형식 반복 출력	★★★★★
CoT	단계별 사고 유도	수학, 논리, 복잡 추론	★★★★★
ReAct	추론·행동·관찰 루프	에이전트 작업, 툴 사용	★★★★☆

KERNE L	영어	핵심 원칙
K	Keep simple	프롬프트를 단순하게 유지하라
E	Explain needs	필요한 것을 명확히 설명하라
R	Refine	결과를 보고 반복적으로 개선하라
N	Narrow	범위를 좁혀서 초점을 맞춰라
E	Explore	다양한 옵션을 탐색하라
L	Learn	예시를 통해 AI가 패턴을 학습하게 하라

할루시네이션 없는 AI 콘텐츠 만들기

AI가 틀린 정보를 자신있게 말하는 현상 — 방지 전략

🚨 할루시네이션 발생 상황

- ▶ 사실 확인 필요한 역사·과학 정보
- ▶ '가장' '최초' '최대' 같은 최상급 표현
- ▶ 특정 시점 기준 수치·통계
- ▶ 코드 API가 실제 존재하는지
- ▶ 외부 라이브러리 버전·메서드명
- ▶ 퀴즈 문제의 정답 정확성

✅ 방지 전략

- ▶ 정답이 하나뿐인지 확인 요청
- ▶ 기준 명시 (2024년 기준, 면적 기준)
- ▶ 2개 이상 출처 교차 검증 요청
- ▶ 논란 내용은 주류 학설 기준 명시
- ▶ 코드는 반드시 실행해서 검증
- ▶ GEMINI.md에 검증 룰 문서화

GEMINI.md 검증 룰

GEMINI.md - 퀴즈 문제 검증 가이드라인

1. 정답이 하나뿐인가? (다른 해석 가능 시 조건 명시)
2. 최상급 표현에 기준이 있는가? ('가장 큰' → 면적 기준)
3. 시간과 범위가 명확한가? (변할 수 있는 정보 시점 명시)
4. 교차 검증했는가? (2개 이상 출처 확인)

고급 프롬프트 전략 — ICL / CoT / ReAct



In-Context Learning (ICL)

Few-shot Prompting

- 학습 없이 예시만 보여줘 동작 유도
- Zero-shot: 예시 없이 지시만
- One-shot: 예시 1개
- Few-shot: 예시 3~5개 → 정확도 ↑

Few-shot 예시

입력: 고객이 화남 → 분류: 부정

입력: 배송이 빠름 → 분류: 긍정

입력: 상품이 마음에 들 → 분류: ?



Chain-of-Thought (CoT)

단계별 사고 유도

- "단계별로 생각해줘" 추가만으로 효과
- 복잡한 수학·논리 문제에 강력
- Zero-shot CoT: "Let's think step by step"
- Few-shot CoT: 풀이 과정 포함 예시 제공

CoT 적용 예

Q: 3명이 일을 나눠서 6일이면

완료. 2명이면 며칠?

→ "단계별로 계산해줘"

Step1: 총 작업량 = $3 \times 6 = 18$ 인일

Step2: 2명이면 $18 \div 2 = 9$ 일



ReAct

Reason + Act

- Reasoning(추론)과 Acting(도구 호출) 결합
- LLM Agent의 핵심 패턴
- Thought → Action → Observation 반복
- Cursor AI / LangChain Agent에서 내부 사용

Thought: 파일 목록을 먼저 확인해야 함

Action: `list_files("./src")`

Observation: `["main.py", "utils.py"]`

Thought: main.py를 읽어야 함

Action: `read_file("main.py")`



Cursor AI는 내부적으로 ReAct 패턴으로 파일 탐색·수정을 수행합니다. 프롬프트에 "단계별로"를 추가하면 CoT가 자동 활성화됩니다.

제로샷 & 퓨샷 프롬프팅

제로샷 (Zero-Shot)

예시 없이 직접 작업 지시
→ 모델이 학습한 지식으로 추론

"이 문장을 영어로 번역해줘:
'오늘 날씨가 정말 좋네요!'"

- ✅ 적합: 명확한 단일 작업
- ❌ 부적합: 복잡한 형식·특정 스타일 요구

퓨샷 (Few-Shot)

2~5개 예시를 함께 제공
→ 패턴 학습 후 동일 형식으로 출력

예시 1: "두 수를 더하는 함수"
→ `def add(a,b): return a+b`

예시 2: "두 수를 곱하는 함수"
→ `def mul(a,b): return a*b`

이제 "두 수의 최댓값 함수"를 작성해줘

- ✅ 적합: 형식·스타일 일관성 필요
- ✅ 코드·문서 생성, 데이터 변환에 강력

생각의 사슬 (CoT) & 2.3.4 페르소나 기법

Chain-of-Thought (CoT)

"step by step으로 생각해"

→ AI가 추론 과정을 단계적으로 서술

"15% 할인 후 20% 추가 할인 총 할인율을 단계별로 계산해"

1단계: $10,000 \times 0.85 = 8,500$ 원

2단계: $8,500 \times 0.80 = 6,800$ 원

∴ 총 32% 할인

- ✓ 수학·논리·복잡한 의사결정에 효과적
- ✓ "Let's think step by step" 문구 추가

페르소나 (Persona) 기법

AI에게 특정 역할·전문성을 부여

→ 일관된 관점·어조·전문 지식으로 응답

 시니어 개발자

"Clean Code 관점에서 리뷰해줘"

 경영 컨설턴트

"McKinsey 방식으로 분석해줘"

 QA 엔지니어

"엣지 케이스를 찾아줘"

 초등학교 선생님

"쉽게 설명해줘"

 ReAct = Reasoning + Acting
Thought → Action → Observation 루프로 자체 교정

사고 제어 기법 (Cognitive Control)

복잡한 문제 해결을 위한 고급 기법

기법	역할	프롬프트 예시
Chain-of-Thought	단계적 추론 유도	'단계별로 생각해서 답해줘'
Few-Shot	예시로 행동 패턴 유도	예시 2-3개 제공 후 질문
Extended Thinking	복잡한 문제에 충분한 시간 부여	'천천히 깊이 생각해줘'
Slow Thinking	가정 검토, 성급한 결론 방지	'결론 내리기 전에 가정을 검토해'
Self-consistency	여러 추론 결과 비교	'3가지 방법으로 풀고 비교해줘'
ReAct	행동-관찰-수정 루프	에이전트 기반 작업 처리

검색 증강 생성 (RAG) — 개요 및 기본 개념

RAG (Retrieval-Augmented Generation)

LLM의 지식 컷오프·환각 문제를 외부 검색으로 보완하는 기술 — 실시간·도메인 특화 정보를 제공



✗ 기존 LLM

학습 데이터 기준 고정 | 최신 정보 없음 | 환각 위험 높음

✓ RAG

실시간 정보 검색 | 출처 제공 | 도메인 특화 가능

RAG 핵심 구성요소 & 실제 적용 사례

 01

Document Loader

PDF·웹페이지·DB에서 텍스트 추출

 02

Text Splitter

긴 문서를 청크(Chunk)로 분할

 03

Embedding Model

텍스트 → 고차원 벡터 변환

 04

Vector Store

벡터 저장 및 유사도 검색
(FAISS·Pinecone)

 05

Retriever

질문과 유사한 문서 청크 추출

 06

LLM + Prompt

검색 결과를 컨텍스트로 최종 답변 생성

실제 적용 사례: Google NotebookLM

- PDF·웹페이지·유튜브를 업로드하면 AI가 내용 기반으로 Q&A 응답 (순수 RAG 구조)
- 환각 최소화: 내 문서 내용만 참조 | 출처 인용 포함 | 오디오 팟캐스트 생성 지원

파인튜닝 (Fine-Tuning)

사전 학습된 대형 모델을 특정 도메인·태스크 데이터로 추가 학습 → 전문화된 성능

전체 파인튜닝

- 모든 파라미터를 재학습
- 최고 성능 달성 가능
- 대규모 GPU·데이터 필요
- 시간·비용 높음
- 적합: GPT-4급 기반 의료·법률 특화 모델

부분 파인튜닝 (LoRA/PEFT)

- 소수 파라미터만 학습 (저랭크 행렬)
- GPU 메모리 90%+ 절감
- 학습 속도 빠름, 성능도 준수
- 원본 모델 보존 가능
- 적합: 스타트업·기업 빠른 프로토타입

파인튜닝 단계 & 활용 사례

파인튜닝 4단계

1

데이터 준비

도메인 특화 데이터 수집·전처리
입력·출력 쌍 구성 (instruction
tuning 형식)

2

기본 모델 선택

GPT-3.5/LLaMA/Gemma 등 오픈소
스 선택
태스크 복잡도·비용 고려

3

학습 실행

하이퍼파라미터 조정 (lr, epoch,
batch)
LoRA 어댑터 또는 전체 파라미터
학습

4

평가 & 배포

벤치마크 평가 → RLHF 추가 개선
API 또는 로컬 배포

활용 사례

의료 챗봇

병원 내부 진료 기록·프로토콜
학습 → 진단 보조

고객 서비스

기업 제품 문서로 학습
→ 자동 CS 응대

법률 분석

판례·계약서 학습
→ 리스크 요약

코드 생성

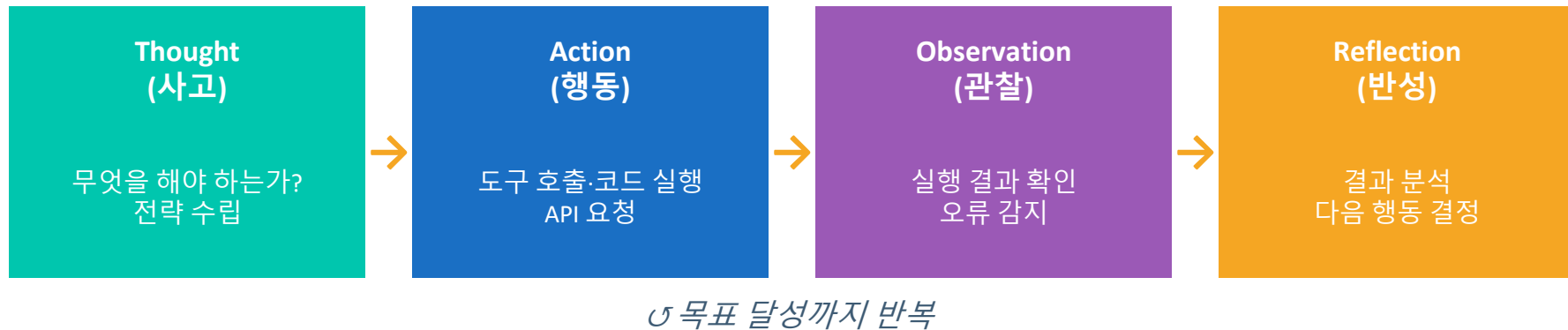
사내 코딩 컨벤션 학습
→ 맞춤형 코드 추천

AI 에이전트 — 정의·원리·사용 사례

AI 에이전트란?

목표를 받으면 스스로 계획·도구 사용·실행·검증을 반복하며 달성하는 자율적 AI 시스템

에이전트 동작 원리 (ReAct Loop)



◆ 코딩 에이전트: Cursor/Cline: 자동 코드 생성→테스트→수정

◆ 업무 자동화: 이메일 분류→CRM 업데이트→일정 조율

◆ 리서치 에이전트: 웹 검색→요약→보고서 자동 작성

◆ MCP 에이전트: DB·API·파일 접근으로 실제 시스템 조작

정리 — RAG vs 파인튜닝 vs 에이전트 비교

구분	RAG	파인튜닝	AI 에이전트
목적	최신 정보 주입	도메인 전문화	자율 태스크 실행
학습 필요	X 불필요	✅ 필요	X 불필요
실시간성	✅ 가능	X 학습 후 고정	✅ 가능
비용	중간	높음	낮음~중간
환각 위험	낮음 (출처 기반)	중간	중간 (도구 오용)
대표 사례	NotebookLM	GPT-4 파인튜닝	Cursor·AutoGPT

🎯 선택 가이드: 최신 정보가 필요하면 → RAG | 특정 스타일·전문성 필요 → 파인튜닝 | 복잡한 자율 작업 → AI 에이전트 | 시스템 연동 → MCP

1일차 강의 자료 · 8시간

워크 북 실습

실습- 실무 프롬프트 고치기 & 개선하기

실습— AI로 나만의 첫 홈페이지 만들기

프롬프트 기반 코딩의 진화

코딩 패러다임의 혁명적 변화

"AI가 코더를 대체하지는 않지만, AI를 활용하는 코더가 그렇지 않은 코더를 대체할 것입니다."

~2023

전통적 코딩

모든 코드를
수작업으로 작성

개발자가 직접
문법·API 암기

2024

AI 보조 코딩

GitHub Copilot,
Tabnine 등

AI가 부분적으로
코드 보완

2025~

AI 에이전트 코딩

Cursor, Cline

AI가 모든 코드를
완전 자동 생성

개발자 = 설계자

기존 개발 vs 바이트 코딩

기존 개발 방식

직접 코드 작성

한 줄씩 타이핑 → 문서 검색 → 스택오버플로우 반복

수동 디버깅

에러 메시지 → 검색 → 원인 추론 → 수정 → 반복

파일별 편집

에디터에서 파일을 열고 닫으며 컨텍스트 전환

긴 피드백 루프

코드 작성 → 빌드 → 테스트 → 수정 cycle이 느림

지식 검색

API 문서, 예제 코드를 직접 찾아보는 시간 낭비

바이트 코딩 방식

의도 중심 지시

"로그인 기능 만들어줘" → AI가 파일 생성·수정 자동화

AI 자율 디버깅

에러 발생 시 AI가 원인 분석 → 수정 → 검증 루프 자동

프로젝트 전체 이해

AI가 전체 코드베이스 파악 후 관련 파일 동시 수정

즉각적 피드백

지시 → 코드 생성 → 실행 → 결과 확인까지 수분 내 완료

검증 중심 역할

개발자는 '무엇을 만들지'에 집중, AI는 '어떻게'를 담당

주요 AI 코딩 에이전트 비교

2025년 기준

도구	특징	강점	가격
Cursor AI	VS Code 기반 IDE, 멀티파일 편집	Agent/Plan/Ask 3모드, .cursorrules	Free / \$20~
GitHub Copilot	GitHub 공식 통합, 모든 IDE 지원	기업 보안 강함, 대형 레포 최적화	\$10~\$19/월
Cline	오픈소스, VS Code 확장	SNS 인기, 직접 API 사용 가능	Free (API 비용)
Amazon Q	AWS 생태계 최적화	AWS 서비스 연동, 기업용	\$19/월
Claude (Anthropic)	Artifacts, Projects 기능	긴 컨텍스트, 문서 작업 특화	Free / \$20~
Gemini CLI	Google	1M 토큰 컨텍스트, Google Drive 연동 , 무료 티어 넉넉	
OpenAI Codex CLI	OpenAI	GPT-4o 기반, 함수 단위 완성 강점, GitHub Copilot 연계	

Cursor AI 요금제 비교 (2025 기준)

가입 즉시 사용 가능, Pro 14일 무료 체험

요금제	가격	자동 완성	프리미엄 요청	비고
Free	무료	월 200~2000회	월 50회 (느림)	2주간 Pro 체험 포함
Pro	\$20/월	무제한	월 500회 (빠름) + 느림 무제한	GPT-4, Claude 사용 가능
Business	\$40/월/인	Pro 기능 전체	Pro + 조직용 보안·대시보드	팀 사용 추천

Cursor AI 주요 단축키

핵심 커맨드 모음

단축키	기능	설명
Tab	코드 자동 완성 수락	약 1초 가만히 있으면 Copilot++ 제안 표시
Cmd + K	인라인 프롬프트	현재 위치에서 AI에게 코드 생성·수정 요청
Cmd + L	AI Chat 패널 열기	우측 사이드바에서 대화형 AI 질문
Cmd + I	Composer (Agent)	전체 프로젝트 범위 멀티파일 작업
@파일명	파일 컨텍스트 참조	특정 파일을 프롬프트에 포함
@심볼	코드 심볼 검색	함수·클래스를 빠르게 찾아 참조

Python 실습 환경 구성

개발 환경 셋업 가이드

▶ Python 설치 (권장: 3.10.11)

- python.org에서 다운로드 → 설치 시 'Add to PATH' 필수 체크
- cmd에서 확인: `python --version`

▶ 가상 환경 생성 (venv 권장)

- `python -m venv py3_10_basic`
- `py3_10_basic\Scripts\activate` (Windows)
- `source py3_10_basic/bin/activate` (Mac)

▶ 필수 라이브러리 설치

- `pip install pytest matplotlib pandas jupyter`
- `pip install PyQt5` (UI 실습용)

▶ IDE 선택

- Cursor AI (권장) | PyCharm Community (무료) | VS Code + Python 확장

소프트웨어 형상 관리(SCM)

Software Configuration Management

SCM 개요

SCM의 목적:

- ① 산출물 품질의 향상
- ② 개발 및 유지보수 생산성 향상
- ③ 사용자 요구사항 체계적 관리
- ④ 변경 추적 및 통보

SCM의 4대 활동:

- ① 형상 식별 (CI)
- ② 형상 통제 (CC)
- ③ 형상 이력관리 (CS)
- ④ 형상 평가 (CA)

버전 관리 시스템 종류

버전 관리 시스템 종류:

로컬 VCS:

단일 PC에서 관리
→ 협업 불가

중앙집중식 VCS:

SVN, CVS
→ 서버 장애 시 전체 중단

분산형 VCS (Git):

각 개발자가 전체 저장소 보유
→ 오프라인 작업 가능
→ 서버 없이 커밋
→ 브랜치 생성/전환 빠름

Git vs GitHub 비교

도구의 역할 이해

구분	Git	GitHub
성격	소프트웨어 (로컬 도구)	웹 서비스 (클라우드 플랫폼)
역할	버전 관리, 변경 추적	원격 저장소, 협업 허브
오프라인	가능 (모든 명령 로컬)	불가능 (인터넷 필요)
유사 서비스	없음 (표준 도구)	GitLab, Bitbucket, Gitea
창시자	Linus Torvalds (2005)	Tom Preston-Werner (2008)

핵심 Git 명령어

실무 필수 명령어 정리

명령어	기능	예시
git init	로컬 저장소 초기화	git init my-project
git clone	원격 저장소 복제	git clone https://github.com/user/repo.git
git add	변경 파일 스테이징	git add . git add main.py
git commit	스냅샷 저장	git commit -m 'feat: 로그인 기능 추가'
git push	원격 저장소에 업로드	git push origin main
git pull	원격 변경사항 통합	git pull origin main
git branch	브랜치 생성/확인	git branch feature-login
git checkout	브랜치 전환	git checkout feature-login

Git 기본 워크플로우

Working Directory → Staging → Local → Remote



git pull / git fetch (원격 → 로컬 동기화)

브랜치 관리 전략

Git Flow / Feature Branch

▶ 기본 브랜치 구조

- main: 배포 가능한 안정 버전
- develop: 개발 통합 브랜치
- feature/xxx: 기능별 개발 브랜치
- hotfix/xxx: 긴급 버그 수정

▶ 실무 워크플로우

- `git branch feature-login` # 브랜치 생성
- `git checkout feature-login` # 브랜치 전환
- ... 개발 ...
- `git add . && git commit -m 'feat: 로그인 구현'`
- `git push origin feature-login`
- # GitHub에서 Pull Request 생성 → 코드 리뷰 → Merge

▶ 커밋 메시지 컨벤션

- feat: 새 기능 | fix: 버그 수정 | docs: 문서
- refactor: 리팩토링 | test: 테스트 | chore: 기타

1.4 Cursor AI 설치 및 환경 점검 · Git 연동

1교시 실습

⚙️ Cursor AI 설치 체크리스트

- ☐ cursor.com 접속 → 다운로드
- ☐ VS Code 스타일 설정 가져오기
- ☐ 기본 모델: claude-3.7-sonnet 선택
- ☐ Cmd+K / Cmd+L / Cmd+I 동작 확인
- ☐ .cursorrules 파일 생성 연습
- ☐ Python 3.10+ & pip 확인
- ☐ git --version 확인
- ☐ GitHub 계정 로그인

🔑 Cursor 3가지 모드

Agent 🤖

복잡한 코딩·리팩토링
다중 파일 자율 수정

Plan 📅

복잡한 작업 계획 수립
코드 수정 없이 계획만

Ask 🗣️

핵심 단축키

Cmd+K 인라인 편집 · Cmd+L Chat 패널
Cmd+I Composer(Agent) · Tab 자동완성 수락

02

아름다운 소프트웨어를 위한 해석학적 접근

Concept-to-Code Traceability · 의사결정 파이프라인 · 문제인식 + 온톨로지 + 프롬프팅 체인

2·3교시

120분

2.1 왜 설계 의도는 코드에서 사라지는가? — Semantic Erosion(의미침식)

2:3교시

"당신의 팀은 완벽한 요구사항 문서를 작성했다. 6개월 후 코드를 보니 설계 의도는 온데간데없고 스파게티만 남아 있다."



문서-코드 단절

개념 분석 산출물이
코드와 직접 연결 안 됨



테스트 수준 혼재

기술 단위 테스트만 작성
비즈니스 의미 검증 없음



UI의 도메인 잠식

UI 요구사항이 도메인
모델을 오염



아키텍처 우선 개발

아키텍처가 개념보다
먼저 결정됨

결과: "Semantic Erosion" — 시스템은 시간이 지날수록 의미적 일관성(Semantic Coherence)을 잃는다

도메인 로직 → UI 이동 · 컨트롤러 비대화 · 테스트가 구현 세부사항만 기술 · 원래 개념 모델 사라짐

2.2 문제인식(Problem Recognition) + 의사결정 파이프라인

2.3교시

문제인식이란: "언제 시스템이 실패했다고 말해야 하는가?"를 정의 — "무엇을 만들까"가 아닌 실패 조건부터

의사결정 파이프라인 전체 흐름



✗ 해결책 중심 (나쁜 문제인식)

- 책을 빌리는 기능이 필요하다
- 로그인 화면을 만들자
- 검색 기능을 추가하자

✓ 실패 조건 중심 (올바른 문제인식)

- 연체·한도 초과 시 → 대출 거부
- 이메일 오류·비밀번호 오류 각각 처리
- 검색어 0자·특수문자·결과 없음 정의

2.3 온톨로지(Ontology) — ECB 분석 모델로 세계를 구조화

2.3교시

온톨로지: "우리가 세계를 어떤 개념과 관계로 나누어 이해할 것인가"를 정의하는 명시적 모델 — 팔란티어의 성공 비결
예: '직원(객체)' 이 'OO 코리아(조직)'에서 근무한다

주요 온톨로지 예시

- 지식 그래프 및 의미망: '김철수(35세, 남, 테슬라 코리아 소속)'와 같이 객체 간의 복합적 관계를 그래프 형태로 표현하여 AI가 맥락을 이해하게 함.
- 분야별 온톨로지: 의학(질병, 증상, 약물 간 관계), 전자상거래(상품, 카테고리, 리뷰 데이터 간 관계), 에너지 산업(유정, 엔지니어, 상태 정보 간 공통 관점).
- 구조적 정의: "A는 B의 일종이다(is-a)", "A는 B를 가지고 있다(has-a)"와 같은 명세서.

<<Entity>>

Entity (엔티티)

시스템이 장기적으로 관리하는 도메인 정보

예: Member · Book · Loan · Order

💡 판단하지 않는다. 상태를 유지한다.

<<Control>>

Control (제어)

유스케이스를 수행하는 흐름 제어·업무 조정

예: LoanPolicy · LoginControl · OrderService

💡 판단한다. 조율한다. 상태는 최소화.

<<Boundary>>

Boundary (경계)

시스템과 외부 액터 사이의 접점

예: LoanUI · LoginAPI · ExternalPayment

💡 비즈니스 규칙을 포함하지 않는다.

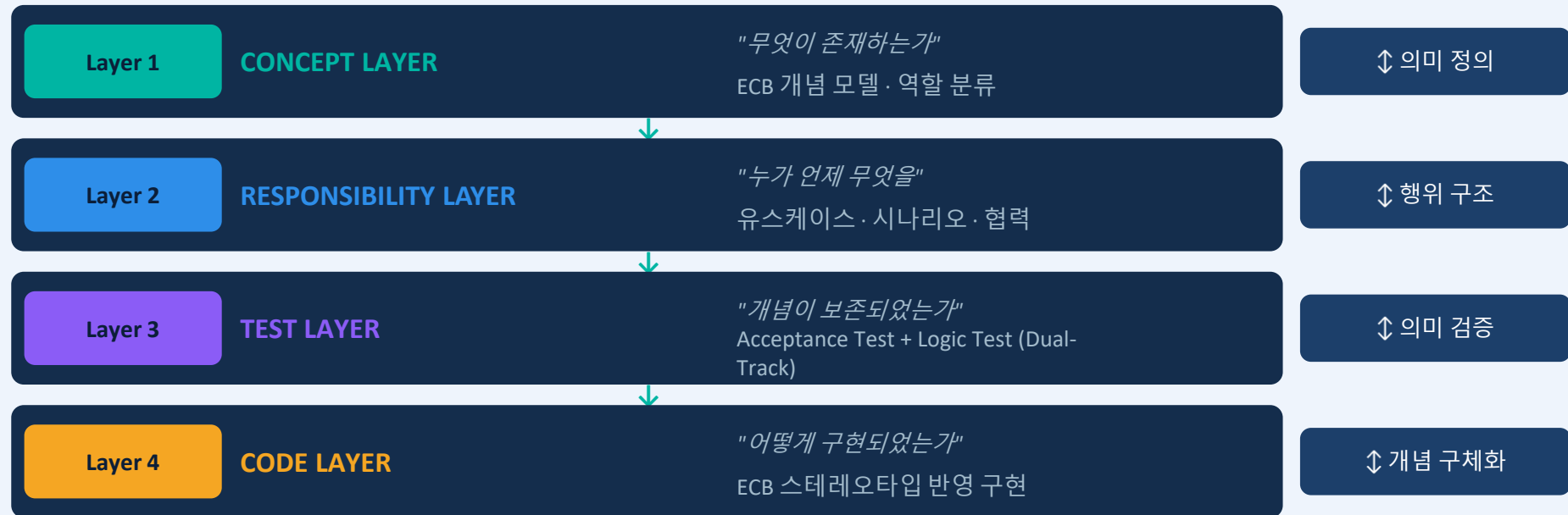
⚠ ECB는 MVC·Spring Controller와 다르다. 분석 개념 도구이지, 코딩 규칙이 아니다 (Ivar Jacobson, OOSE 1992)

2.4 Concept-to-Code Traceability — 정의와 4계층 모델

2.3교시

C2C Traceability 정의

문제 영역에서 식별된 개념(Concept)과 코드(Code) 사이의 의미적·구조적·행위적 연결이 명시적으로 유지·검증·진화 가능한 상태 — 의미 보존(Semantic Preservation) 목표



핵심 성질: 완전성(Completeness) · 일관성(Consistency) · 검증 가능성(Verifiability) · 진화 가능성(Evolvability)

2.5 프롬프팅 체인 — C2C Traceability의 현대적 구현

2.3교시

핵심 통찰: 프롬프트 엔지니어링 = C2C Traceability를 구현하는 현대적 방법론

프롬프트 = 명시적 의도 기록(Traceability Documentation) + 맥락 보존(Context Preservation) + 재생성 가능성(Reproducibility)

Layer 1

ECB 개념 분석

ECB 분석: 도서관 시스템
ENTITIES: Member, Book, Loan
CONTROLS: LoanPolicy, LoanService
BOUNDARIES: LoanUI, LoanAPI
각 개념의 ECB 스테레오타입과 책임을 정의하라

Layer 2

책임 시나리오

LoanPolicy(Control)의 책임:
1. 대출 가능 여부 판단
2. 대출 기간 계산
Trace: LoanPolicy는 Concept Layer의 Control

Layer 3

테스트 명세

LoanPolicy.can_borrow()
테스트:
- 정회원 + 0권 → True
- 정회원 + 3권 → False
Trace: Responsibility Layer의 "대출 가능 여부 판단"

Layer 4

코드 구현

위 테스트를 통과하는 LoanPolicy 구현
ECB: <<control>>
Trace: Test→Responsibility →Concept

각 프롬프트가 상위 계층을 명시적으로 참조 → Chain-of-Prompts = Traceability Chain → Traceability 자동 확보

2.6 ECB(Entity-Control-Boundary) 분석 패턴

개체 역할 기반 설계

E

Entity 엔티티

시스템이 장기적으로 보관해야 하는 데이터 정보

특징:

- 데이터 보유 (속성)
- 비즈니스 규칙 포함
- DB 테이블과 매핑

예시:

User, Product, Order
BookInfo, Inventory

B

Boundary 경계

시스템과 외부 액터(사람·시스템) 사이의 인터페이스

특징:

- UI 화면, API 엔드포인트
- 입출력 검증
- 외부와의 통신

예시:

LoginForm, ApiController
DashboardView, WebSocket

C

Control 제어

유스케이스 흐름을 조정하는 비즈니스 로직

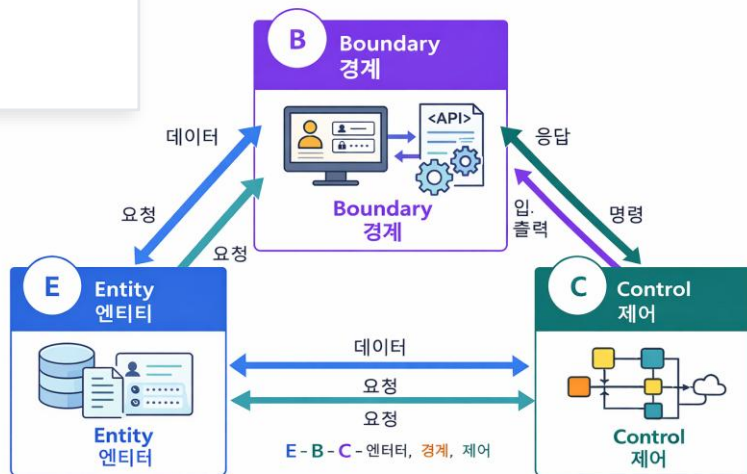
특징:

- Entity와 Boundary 연결
- 비즈니스 규칙 실행
- 트랜잭션 관리

예시:

AuthService, OrderProcessor
BookSearchController

ECB 모델



ECB에서 Control이 비즈니스 규칙을 실행하고 Boundary가 외부와 통신한다면 — RBAC는 "누가 어떤 Control을 실행할 수 있는가"를 결정하는 접근 제어 레이어입니다

ECB 패턴에 RBAC를 더하면

- E Entity**
User·Role·Permission·Resource
→ RBAC 핵심 데이터 보관
- B Boundary**
로그인 폼·API 엔드포인트
→ Role 정보를 받아 Control로 전달
- C Control**
AuthService·PermissionChecker
→ Role-Permission 매핑·접근 허용/거부

RBAC 4대 구성 요소

- User** 시스템 사용자 — 개발자·운영자·관리자
- Role** 권한 묶음 단위 — admin·developer·viewer
- Permission** 자원에 대한 행동 — read·write·delete
- Resource** 접근 대상 — DB·API·파일·서비스

용어 구분: 큰 틀 vs 세부 표현

- 큰 틀** → 역할 기반 접근 제어(RBAC)
- 메뉴·UI** → 권한(역할)에 따른 메뉴 가시성 제어
- 내비게이션** → RBAC에 따른 내비게이션 제어

"권한에 따른 메뉴 가시성 제어" = Boundary(UI)가 Control(AuthService)에 Role을 질의해 메뉴를 보여줄지 숨길지 결정 — UX Contract의 '보인다/안 보인다' 언어와 직결

Role-Permission 매트릭스 (내비게이션 메뉴 기준)

역할	대시보드	사용자관리	설정	삭제
admin	✓	✓	✓	✓
developer	✓	✓	✗	✗
viewer	✓	✗	✗	✗
guest	⚠	✗	✗	✗

✓ UX Contract 연결 — "보인다 / 안 보인다" 규칙

Role=admin:	사용자관리 메뉴 보인다 · 삭제 버튼 활성화
Role=developer:	사용자관리 메뉴 보인다 · 삭제 버튼 비활성화
Role=viewer:	대시보드만 보인다 · 모든 관리 메뉴 숨김

Python — 메뉴 가시성 제어 구현

```
from enum import Enum

class Role(Enum):
    ADMIN = 'admin'
    DEVELOPER = 'developer'
    VIEWER = 'viewer'

# 역할별 메뉴 가시성 정의
MENU_VISIBILITY = {
    Role.ADMIN: {
        'dashboard': True,
        'user_mgmt': True,
        'settings': True,
        'delete_btn': True,
    },
    Role.DEVELOPER: {
        'dashboard': True,
        'user_mgmt': True,
        'settings': False,
        'delete_btn': False,
    },
    Role.VIEWER: {
        'dashboard': True,
```

R

RBAC (Role-Based Access Control) — 역할 기반 접근 제어

Superadmin, Admin, Manager, User, Viewer 등 역할에 따라 기능 접근을 제어

예: Admin만 사용자 관리 가능, Viewer는 읽기만 가능

권한(permission) 기반 작업 시 유지보수 수월

T

TBAC (Team-Based Access Control) — 팀 기반 접근 제어

사용자가 소속된 팀에 따라 데이터 접근 범위를 제어

예: R&D 팀 소속 사용자는 R&D 팀 게시판/문서만 접근 가능

A

ABAC (Attribute-Based Access Control, 속성 기반 접근 제어)라고도 합니다. 팀 소속이라는 "속성"을 기준으로 접근 제어

RBAC를 TDD로 구현할 때 Cursor AI를 활용하는 전체 흐름 — C2C Traceability: 역할 개념 → 실패 테스트 → 최소 구현 → 리팩토링까지 의미 소실 없이 추적



RED 단계 — 실패 테스트 예시

```
def test_viewer_cannot_see_user_mgmt():    # 메뉴 가시성
    assert not is_visible(Role.VIEWER, 'user_mgmt')
```

```
def test_admin_can_see_all_menus():        # admin 전체 접근
    assert all(is_visible(Role.ADMIN, m) for m in
MENU_VISIBILITY[Role.ADMIN])
```

용어 정리

- ▶ 역할 기반 접근 제어(RBAC)
- ▶ 권한(역할)에 따른 메뉴 가시성 제어
- ▶ RBAC에 따른 내비게이션 제어

04

Magic Square × C2C × 문제인식

D1_4.실습- 간단한 앱 설계 (3계층)

마방진으로 배우는 Concept-to-Code Traceability 실전 적용

4교시

60분

4.1 Magic Square (마방진) 프로젝트 소개

4교시

Magic Square = 모든 행·열·대각선의 합이 같은 4x4 숫자 배열 — 두 빈칸 위치를 찾고 1~16 숫자 배치

4x4 Magic Square

16	3	2	13
5	10	11	?
9	6	?	12
4	15	14	1

합 = 34

ECB 개념 분석 (문제인식 → 설계)

<<Entity>>

MagicSquare — 4x4 격자 데이터 · Cell — 각 칸 값·위치·상태 · SolveResult — 해결 결과

<<Control>>

SquareValidator — 합 유효성 검증 · MissingFinder — 빈칸 위치 탐색 · Solver — 완성 알고리즘

<<Boundary>>

GridUI — 격자 시각화 · InputHandler — 사용자 입력 · ResultDisplay — 결과 표시

실습 목표: ECB 분류 완성 + 문제인식(실패 조건) 3가지 이상 도출

4.2 Magic Square — 문제인식 & C2C 연결 실습

4교시 실습

"해결책이 아닌 실패 조건을 먼저 정의하라" — 아래 표를 완성하는 것이 4교시 핵심 실습입니다

개념 (ECB)	책임 (Responsibility)	실패 조건 (Problem Recognition)	테스트 힌트
SquareValidator <<control>>	행·열·대각선 합 검증	합이 34가 아닐 때 → 유효하지 않은 배열	<code>is_valid() == False</code>
MissingFinder <<control>>	빈칸 위치 탐색	빈칸이 0개 또는 3개 이상 → 탐색 불가	<code>find_missing() raise</code>
MagicSquare <<entity>>	격자 데이터 보관	1~16 범위 초과 값 → 데이터 무결성 위반	<code>valid_range() == False</code>
GridUI <<boundary>>	격자 시각화 및 입력	입력값이 정수가 아닐 때 → UI 거부	<code>validate_input()</code>

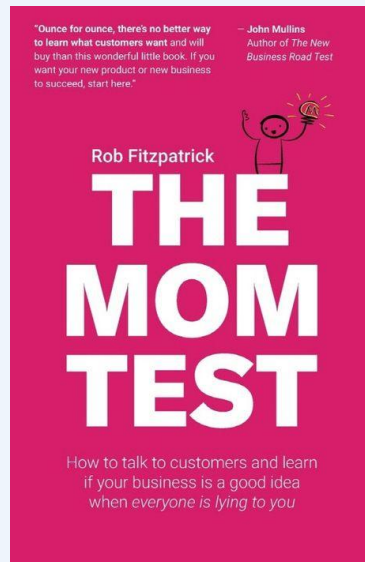
실습 과제: 위 표를 완성하고 "빈칸이 대각선에만 있을 때" 추가 실패 조건을 스스로 도출해보세요

4.3 The Mom Test 란?

4교시 실습

The Mom Test는 스타트업/제품 아이디어 검증에서 쓰이는 인터뷰 기법으로, 상대가 '좋은 사람이라서' 거짓 긍정(칭찬, 공감)을 해도 속지 않도록 질문하는 방법.

Mom Test = "사람의 말이 아니라 행동을 믿는 인터뷰 방법"



The Mom Test & 문제 인식 훈련

잘못된 질문의 예:

❌ "제가 만든 설비 모니터링 앱 어때요?"
→ 칭찬만 들을 것

❌ "이런 기능이 있으면 사용하시겠어요?"
→ 가짜 약속만 받을 것

❌ "이 앱에 얼마 지불하시겠어요?"
→ 현실성 없는 답변만 받을 것

The Mom Test & 문제 인식 훈련

올바른 질문의 예:

- ✓ "최근에 설비 점검하면서 가장 답답했던 순간이 언제였나요?"
→ 진짜 문제 발견
- ✓ "지금은 그 상황을 어떻게 해결하고 계세요?"
→ 현재 우회 방법 파악
- ✓ "그 방법의 어떤 점이 불편하세요?"
→ 개선 포인트 확인

The Mom Test & 문제 인식 훈련

핵심 원칙 3가지:

당신의 아이디어에 대해 말하지 마라
미래나 가정에 대해 묻지 마라
과거의 구체적인 행동에 대해 물어라

잘못된 데이터 피하기

피해야 할 3가지:

칭찬 (Compliments)

- 사용자: "오, 이거 정말 좋은 아이디어네요!"
- → 의미 없음. 행동으로 이어지지 않음
- 대신 물어볼 것:
- "비슷한 문제를 겪으신 적 있으세요? 그때 어떻게 하셨나요?"

가상의 약속 (Hypotheticals)

- 사용자: "그런 기능 있으면 꼭 쓸 것 같아요!"
- → 실제로는 안 씀
- 대신 물어볼 것:
- "현재 유사한 도구를 사용해보신 적 있나요? 얼마나 자주 쓰시나요?"

| 잘못된 데이터 피하기

일반론 (Generics)

- 사용자: "보통 사람들은 이런 거 좋아할 거예요"
- → 추측일 뿐

- 대신 물어볼 것:
- "선생님께서서는 직접 어떻게 하시나요?"

올바른 질문하기

좋은 질문의 특징:

과거의 구체적 사건

- "지난주에 설비 점검하실 때 어떤 일이 있었나요?"
- "그때 정확히 어떤 절차를 거치셨나요?"
- "얼마나 시간이 걸렸나요?"

돈/시간 투자 확인

- "현재 사용 중인 도구에 비용을 지불하고 계신가요?"
- "그 도구를 배우는 데 얼마나 시간을 쓰셨나요?"
- "팀원들도 다 쓰고 있나요?"

올바른 질문하기

해결 시도 이력

- "이 문제를 해결하려고 시도해본 적 있으세요?"
- "어떤 도구들을 써보셨나요?"
- "왜 계속 쓰지 않으셨나요?"

4.4 실습 과제 전체 가이드

[해결할 문제]

- 마방진(Magic Square)이란 가로, 세로, 대각선 방향의 수를 더한 값이 모두 같은 정사각형 행렬입니다. 마방진에는 `1`부터 `정사각형 넓이`까지, 수가 하나씩 배치되어야 합니다. 아래는 가로, 세로, 대각선 방향의 수를 더한 값이 모두 34인 4 x 4 마방진입니다.

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

1. 두 빈칸의 위치를 찾습니다.
2. 숫자 1 ~ 16 중 존재하지 않는 숫자 2개를 찾습니다.
3. 첫 번째 빈칸에 작은 숫자를, 두 번째 빈칸에 큰 숫자를 넣어 행렬이 마방진이 되는 지 검사합니다.
4. return
 - 4-1. 마방진이라면 [작은 숫자의 행 번호, 작은 숫자의 열 번호, 작은 숫자, 큰 숫자의 행 번호, 큰 숫자의 열 번호, 큰 숫자]를 return 합니다.
 - 4-2. 마방진이 아니라면 [큰 숫자의 행 번호, 큰 숫자의 열 번호, 큰 숫자, 작은 숫자의 행 번호, 작은 숫자의 열 번호, 작은 숫자]를 return 합니다.

4.4 실습 과제 전체 가이드

```
package com.samsung;

import java.util.ArrayList;
import java.util.Arrays;

class Pair {
    Integer y;
    Integer x;

    public Pair(Integer y, Integer x) {
        this.y = y;
        this.x = x;
    }

    public Integer first() {
        return y;
    }

    public Integer second() {
        return x;
    }
}
```

```
class MagicSquare {
    public static final int n = 4;

    // 빈칸(0)의 좌표를 찾는 메서드
    public ArrayList<Pair> find_blank_coords(int[][] matrix) {
        ArrayList<Pair> coords = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    coords.add(new Pair(i, j));
                }
            }
        }
        return coords;
    }
}

...
```


4.4 실습 과제 전체 가이드

// 존재하지 않는 숫자를 찾는 메서드

```
public ArrayList<Integer> find_not_exist_nums(int[][] matrix) {
    ArrayList<Integer> nums = new ArrayList<>();
    boolean[] exist = new boolean[n * n + 1];
    Arrays.fill(exist, false);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] != 0) {
                exist[matrix[i][j]] = true;
            }
        }
    }

    for (int i = 1; i <= n * n; i++) {
        if (!exist[i]) {
            nums.add(i);
        }
    }

    return nums;
}
```

// 마방진인지 확인하는 메서드

```
public boolean is_magic_square(int[][] matrix) {
    int sum = n * (n * n + 1) / 2; // 마방진의 합

    // 행, 열 합 확인
    for (int i = 0; i < n; i++) {
        int row_sum = 0, col_sum = 0;
        for (int j = 0; j < n; j++) {
            row_sum += matrix[i][j];
            col_sum += matrix[j][i];
        }
        if (row_sum != sum || col_sum != sum) {
            return false;
        }
    }

    // 대각선 합 확인
    int main_diagonal_sum = 0, skew_diagonal_sum = 0;
    for (int i = 0; i < n; i++) {
        main_diagonal_sum += matrix[i][i];
        skew_diagonal_sum += matrix[i][n - 1 - i];
    }

    return main_diagonal_sum == sum && skew_diagonal_sum == sum;
}
```

4.4 실습 과제 전체 가이드

// 마방진이 되는지 확인하는 솔루션 메서드

```
public int[] solution(int[][] matrix) {  
    int[] answer = new int[6];  
    int ans_idx = 0;
```

// 1. 빈칸(0)의 좌표 찾기

```
ArrayList<Pair> coords = find_blank_coords(matrix);
```

// 2. 존재하지 않는 숫자 찾기

```
ArrayList<Integer> nums = find_not_exist_nums(matrix);  
nums.sort(Integer::compareTo); // 작은 숫자 먼저
```

// 3. 첫 번째 조합: 작은 숫자 -> 첫 번째 빈칸, 큰 숫자 -> 두 번째 빈칸

```
matrix[coords.get(0).first()][coords.get(0).second()] = nums.get(0);  
matrix[coords.get(1).first()][coords.get(1).second()] = nums.get(1);
```

```
if (is_magic_square(matrix)) {  
    // 마방진이 되는 경우  
    for (int i = 0; i < 2; i++) {  
        answer[ans_idx++] = coords.get(i).first() + 1; // 행 번호 (1-indexed)  
        answer[ans_idx++] = coords.get(i).second() + 1; // 열 번호 (1-indexed)  
        answer[ans_idx++] = nums.get(i); // 숫자  
    }  
} else {  
    // 4. 두 번째 조합: 큰 숫자 -> 첫 번째 빈칸, 작은 숫자 -> 두 번째 빈칸  
    matrix[coords.get(0).first()][coords.get(0).second()] = nums.get(1);  
    matrix[coords.get(1).first()][coords.get(1).second()] = nums.get(0);  
    for (int i = 0; i < 2; i++) {  
        answer[ans_idx++] = coords.get(1-i).first() + 1; // 행 번호 (1-indexed)  
        answer[ans_idx++] = coords.get(1-i).second() + 1; // 열 번호 (1-indexed)  
        answer[ans_idx++] = nums.get(i); // 숫자  
    }  
}  
  
return answer;  
}
```

4.4 실습 과제 전체 가이드

```
public class MagicSquareMain {  
    public static void main(String[] args) {  
        MagicSquare magicSquare = new MagicSquare();  
  
        // 입력 행렬  
        int[][] matrix = {  
            {16, 2, 3, 13},  
            {5, 11, 10, 0},  
            {9, 7, 6, 12},  
            {0, 14, 15, 1}  
        };  
  
        // 결과 출력  
        int[] ret = magicSquare.solution(matrix);  
        System.out.println("결과 값은 " + Arrays.toString(ret) + " 입니다.");  
    }  
}
```

4.4 실습 과제 전체 가이드

0으로 표시된 위치와 채워져야 하는 값을 리턴합니다.

matrix	return
[[16,2,3,13],[5,11,10,0],[9,7,6,12],[0,14,15,1]]	[4,1,4,2,4,8]

4.4 실습 과제 전체 가이드

1. 테스트 코드 작성

(1) find_blank_coords() 메서드 테스트

- 목표: 행렬에서 0 값을 가진 좌표를 정확히 반환하는지 검증.
- 테스트 항목:
 - 빈칸이 여러 개일 때 올바른 좌표 반환.
 - 빈칸이 없는 경우 빈 리스트 반환.

(2) find_not_exist_nums() 메서드 테스트

- 목표: 1~16의 숫자 중 행렬에 없는 숫자를 올바르게 반환하는지 검증.
- 테스트 항목:
 - 일부 숫자가 누락된 경우 올바른 숫자 리스트 반환.
 - 모든 숫자가 존재하는 경우 빈 리스트 반환.

(3) is_magic_square() 메서드 테스트

- 목표: 행렬이 마방진인지 여부를 정확히 판단하는지 검증.
- 테스트 항목:
 - 가로, 세로, 대각선 합이 모두 같은 경우 true 반환.
 - 합이 일치하지 않는 경우 false 반환.

(4) solution() 메서드 테스트

- 목표: 두 개의 빈칸에 숫자를 넣어 마방진을 완성하거나 실패하는 결과를 올바르게 반환.
- 테스트 항목:
 - 빈칸이 2개이며 숫자를 채웠을 때 마방진이 완성되는 경우.
 - 숫자를 어떤 방식으로 채워도 마방진이 되지 않는 경우.
 - 빈칸이 없는 경우.
 - 행렬이 이미 마방진인 경우.

4.4 실습 과제 전체 가이드

JUnit 기반 테스트 코드 예시

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.Arrays;

class MagicSquareTest {

    MagicSquare magicSquare = new MagicSquare();

    @Test
    void testFindBlankCoords() {
        int[][] matrix = {
            {16, 2, 3, 13},
            {5, 11, 10, 0},
            {9, 7, 6, 12},
            {0, 14, 15, 1}
        };
    }
```

```
ArrayList<Pair> result = magicSquare.find_blank_coords(matrix);
assertEquals(2, result.size());
assertEquals(new Pair(1, 3).first(), result.get(0).first());
assertEquals(new Pair(1, 3).second(), result.get(0).second());
assertEquals(new Pair(3, 0).first(), result.get(1).first());
assertEquals(new Pair(3, 0).second(), result.get(1).second());
}
```

```
@Test
void testFindNotExistNums() {
    int[][] matrix = {
        {16, 2, 3, 13},
        {5, 11, 10, 8},
        {9, 7, 6, 12},
        {4, 14, 15, 1}
    };

    ArrayList<Integer> result = magicSquare.find_not_exist_nums(matrix);
    assertEquals(Arrays.asList(8), result);
}
```

4.4 실습 과제 전체 가이드

JUnit 기반 테스트 코드 예시

@Test

```
void testIsMagicSquare() {  
    int[][] magicMatrix = {  
        {16, 2, 3, 13},  
        {5, 11, 10, 8},  
        {9, 7, 6, 12},  
        {4, 14, 15, 1}  
    };  
  
    assertTrue(magicSquare.is_magic_square(magicMatrix));  
  
    int[][] nonMagicMatrix = {  
        {16, 2, 3, 13},  
        {5, 11, 10, 0},  
        {9, 7, 6, 12},  
        {0, 14, 15, 1}  
    };  
  
    assertFalse(magicSquare.is_magic_square(nonMagicMatrix));  
}
```

@Test

```
void testSolution() {  
    int[][] matrix = {  
        {16, 2, 3, 13},  
        {5, 11, 10, 0},  
        {9, 7, 6, 12},  
        {0, 14, 15, 1}  
    };  
  
    int[] result = magicSquare.solution(matrix);  
    assertEquals(6, result.length);  
    assertEquals(new int[]{2, 4, 8, 4, 1, 4}, result);  
}
```

4.4 실습 과제 전체 가이드

2. 리팩토링 항목

(1) Pair 클래스

- 현재 Pair 클래스는 행렬의 좌표를 표현하기 위해 사용됩니다.
- 문제점:
 - 클래스 이름이 좌표를 명확히 표현하지 않음.
 - 메서드 이름(first()와 second())이 의미가 모호함.
- 리팩토링 방안:
 - 클래스 이름을 Coordinate로 변경.
 - 메서드 이름을 getRow()와 getColumn()으로 변경하여 명확히 표현.

4.4 실습 과제 전체 가이드

2. 리팩토링 항목

(1) Pair 클래스

```
class Coordinate {  
    private Integer row;  
    private Integer col;  
  
    public Coordinate(Integer row, Integer col) {  
        this.row = row;  
        this.col = col;  
    }  
  
    public Integer getRow() {  
        return row;  
    }  
  
    public Integer getColumn() {  
        return col;  
    }  
}
```

4.4 실습 과제 전체 가이드

2. 리팩토링 항목

(2) 상수화

- $n = 4$ 와 같은 매직 넘버를 사용하고 있습니다.
- 리팩토링 방안:
 - n 을 MATRIX_SIZE로 상수화하여 의미를 명확히 합니다.

```
public static final int MATRIX_SIZE = 4;
```

4.4 실습 과제 전체 가이드

2. 리팩토링 항목

(3) 반복 코드 제거

- 행렬의 합을 계산하는 코드에서 행과 열의 합을 각각 계산하는 반복문이 중복됩니다.
- 리팩토링 방안:
 - 행과 열 합 계산 로직을 메서드로 추출.

4.4 실습 과제 전체 가이드

2. 리팩토링 항목

(3) 반복 코드 제거

```
private boolean checkSum(int[][] matrix, int sum) {  
    for (int i = 0; i < MATRIX_SIZE; i++) {  
        int rowSum = 0, colSum = 0;  
        for (int j = 0; j < MATRIX_SIZE; j++) {  
            rowSum += matrix[i][j];  
            colSum += matrix[j][i];  
        }  
        if (rowSum != sum || colSum != sum) {  
            return false;  
        }  
    }  
    return true;  
}
```

4.4 실습 과제 전체 가이드

프로젝트 : Magic Square

본인 번호로 Repository 생성, 구현 진행

- Repository : 프로젝트명_개인번호 (ex: Magic_Square_xx)
- description : Author, Reviewer 성명
 - ex: Author : 홍길동, Reviewer : 다자바
- dev branch를 생성 하여 진행
- Commit : 상세한 commit
- Push/Pull Request/Review 진행하세요.

실습 평가 기준 (Magic Square 프로젝트)

평가 항목	배점	세부 기준	도구
테스트 (TDD)	40점	RED→GREEN→REFACTOR 사이클 준수	pytest
코드 품질	30점	SOLID, 클린코드, 스멜 제거	pylint
PRD/Gherkin	20점	시나리오 완성도, 추적성	문서
Git 관리	10점	커밋 단위, 메시지, 브랜치	GitHub
보너스	+10점	CI/CD GitHub Actions 구성	Actions

테스트 자동화 ROI 분석

+15~35%

TDD 개발시간
증가

-40~90%

결함률
감소

-50%

유지보수
비용 감소

-60%

버그 수정
비용 절감

-40%

온보딩
시간 단축

-30%

코드 리뷰
시간 절감

TDD 실습 시 자주 겪는 문제와 해결법

- ▶ ImportError: 모듈 없음 → PYTHONPATH 설정 또는 conftest.py에 sys.path 추가
- ▶ 테스트가 항상 통과: 항상 True assertion 여부 확인, RED 먼저 확인
- ▶ Mock이 작동 안 함: unittest.mock.patch 경로 확인 (import된 경로 기준)
- ▶ 커버리지가 낮음: --cov-branch 옵션 추가, 엡지 케이스 테스트 보완
- ▶ GREEN 단계에서 과도한 구현: '지금 실패하는 테스트만' 원칙으로 복귀
- ▶ REFACTOR 후 테스트 실패: 기능 변경 금지 원칙 재확인, git diff로 검토
- ▶ flake8 오류 폭증: .flake8 설정 파일에 max-line-length=120 등 조정

실습 최종 과제 - Magic Square 완성 요구사항

❌ Bad Code

```
# 제출 구조
magic_square_project/
├── magic_square.py    # 구현 코드
├── test_magic.py      # pytest 테스트
├── PRD.md             # Gherkin 시나리오
├── ECB_diagram.md     # ECB 모델
├── .cursorrules       # AI 규칙
├── .github/
│   └── workflows/
│       └── ci.yml     # GitHub Actions
└── README.md         # 실행 방법
```

✅ Good Code

- # 평가 기준
 - ✅ TDD (40점)
 - RED 실패 테스트 5개 이상
 - GREEN 모두 통과
 - REFACTOR 1개 이상
 - ✅ 코드 품질 (30점)
 - pylint 8.0 이상
 - mypy 오류 0
 - 함수 20줄 이하
 - ✅ PRD/Gherkin (20점)
 - 3개 이상 시나리오
 - Given/When/Then 형식
 - ✅ Git (10점)
 - 의미있는 커밋 메시지
 - 브랜치 전략 사용

핵심 메시지 4가지

01 테스트 없으면 코드 없다

RED를 먼저 써야 GREEN이 의미있다

02 동작하게, 올바르게, 빠르게

GREEN → SOLID 원칙 → 성능 최적화 순서

03 리팩토링은 기능 추가가 아니다

외부 동작 유지 + 내부 구조 개선 = 리팩토링

04 AI와 협력하여 설계하라

Cursor AI는 구현 보조 도구, 설계는 개발자가

1일차 강의 자료 · 8시간

워크 북 실습

실습- The Mom Test + 문제 10개 찾기

실습- 간단한 앱 설계 (3계층)



1일차 강의 자료 · 8시간

워크 북 실습

The Mom Test 를 통해 찾은 **진짜** 문제는 무엇인가?

퍼즐 완성이 아니라 합의된 제약 체계를 만족하는 배치 존재
문제

05

Dual-Track UI + Logic TDD 개발 방법론과 요구사항

왜 우리는 항상 늦게 실패하는가? · Problem & Scenarios · Toy Project

5교시

60분

5.1 왜 우리는 항상 늦게 실패하는가? — 요구사항의 구조적 함정

5교시

"실패 조건이 정의되지 않으면, 성공은 합의될 수 없다"

기획서에는 기능이 있지만 실패 조건이 없다. 테스트는 항상 구현 후에 추가된다. 이것이 "늦은 실패"의 본질이다.

원인 1

기획은 있는데 설계는 없다

요구사항 문서 = 기능 목록
"실패 조건"이 없어 개발자마다 해석이
다름
→ 구현 후 "이게 아닌데" 반복

원인 2

테스트는 항상 늦게 추가된다

"구현 먼저, 테스트는 나중에"
→ 테스트가 구현의 하인이 됨
→ 비즈니스 의미를 잃음

원인 3

UI/Logic 분리의 오해

"UI 따로, 로직 따로" 개발
→ UI가 비즈니스 규칙 포함
→ 어느 것도 독립 테스트 불가

전통적 방식: 기획서→개발자 해석→구현→문제 발견 VS Dual-Track TDD: 실패 조건 합의→RED 테스트→구현

5.2 PRD Is the Code — Problem Recognition & Scenario (L0~L3)

5교시

"PRD는 문서가 아니라 테스트의 원천(Source of Truth)이다" — Problem + Scenario(L0~L3) + UX Contract + Logic Rule

PRD의 핵심 = 기능 목록이 아니라 실패 조건 목록

✗ 해결책 중심 PRD

- 로그인 기능을 만들자
- 책 검색 기능 추가
- 대출 한도 관리

✓ 실패 조건 중심 PRD

- 이메일 형식 오류 → 즉시 에러 표시
- 대출 3권 초과 → 거부 + 이유
- 연체 중인 회원 → 모든 대출 차단

② Scenario Level 0~3 — 실패를 시간축에 배열하다

L0

기능 개요

"회원이 책을 빌린다"

L1

Happy Path

정회원 + 가용 책 → 대출 성공

L2

Edge Case

대출 2권 → 성공 / 3권 → 거부

L3

Failure Path

연체 중 시스템 오류 권한 없음

5.2 PRD Is the Code — Problem Recognition & Scenario (L0~L3)

5교시

코딩 전 AI가 정확하게 실행할 수 있도록 명세서를 먼저 작성

PRD가 필요한 이유

- ▶ 방향성 없는 코딩 방지
- ▶ 명확한 기능 범위 설정
- ▶ 재작업 횟수 70% 감소
- ▶ 단계별 구현 계획 수립
- ▶ 팀 협업 공통 기반 제공

PRD 구성 요소

- ① 프로젝트 개요 (목적·배경)
- ② 타깃 사용자 정의
- ③ 핵심 기능 (Must/Should)
- ④ 기술 스택·제약 조건
- ⑤ 성공 지표 (KPI)

프롬프트

포트폴리오 웹페이지를 만들려고 해.
주요 기능은 채용 담당자가 30초 안에 내
역량을 파악할 수 있는 임팩트 있는 구성
이야.
프리랜서 전환을 준비하는 마케터를 위
한 것이고, 경쟁력 있는 지원자로 보이기
위해 필요해.
기술적인 방향과 함께 구체적인 PRD를
작성해 줘.

📌 실습 순서: PRD 작성 → 단계별 분해 → 단계별 실행 → 검토 요청 → 개선 요청 (5단계 패턴)

5.3 Toy Project 소개 — Magic Square 완전 구현

5교시

Toy Project: Magic Square (4×4 마방진) — 오늘 8교시를 통해 문제인식→PRD→코드까지 완성하는 학습 프로젝트

4교시

문제인식
& ECB 분석

실패 조건 도출
ECB 분류

5교시

Problem &
Scenarios

L0~L3 시나리오
UX/Logic 분리

6교시

Dual-Track
요구사항

UX Contract
Logic Rule 표

7교시

PRD 작성
+ 개발 시작

Gherkin 기반
Cursor AI 활용

8교시

To-Do 구조
작성

체크박스 속성
검증 포인트



최종 목표: Claude Desktop 또는 PyQt5 UI + pytest 통과 + GitHub Push — C2C Traceability 완전 구현

1일차 강의 자료 · 8시간

워크 북 실습

실습 — PRD(제품 요구사항 문서) 작성하기



실습 — PRD(제품 요구사항 문서) 작성하기

코딩 전 AI가 정확하게 실행할 수 있도록 명세서를 먼저 작성

PRD가 필요한 이유

- ▶ 방향성 없는 코딩 방지
- ▶ 명확한 기능 범위 설정
- ▶ 재작업 횟수 70% 감소
- ▶ 단계별 구현 계획 수립
- ▶ 팀 협업 공통 기반 제공

PRD 구성 요소

- ① 프로젝트 개요 (목적·배경)
- ② 타겟 사용자 정의
- ③ 핵심 기능 (Must/Should)
- ④ 기술 스택·제약 조건
- ⑤ 성공 지표 (KPI)

프롬프트

포트폴리오 웹페이지를 만들려고 해.
주요 기능은 채용 담당자가 30초 안에 내
역량을 파악할 수 있는 임팩트 있는 구성
이야.
프리랜서 전환을 준비하는 마케터를 위
한 것이고, 경쟁력 있는 지원자로 보이기
위해 필요해.
기술적인 방향과 함께 구체적인 PRD를
작성해 줘.

📌 실습 순서: PRD 작성 → 단계별 분해 → 단계별 실행 → 검토 요청 → 개선 요청 (5단계 패턴)

단계별 프롬프트 전략

큰 작업을 5단계로 쪼개면 AI가 더 정확하게 실행한다

1

구조 설계

파일 구조·아키텍처만
먼저

2

핵심 기능

비즈니스 로직 구현

3

UI/UX

디자인·스타일·인터랙
션

4

고급 기능

검색·다크모드·단축키

5

최적화

성능·접근성·에러 처리

💡 단계 분해 프롬프트: "이 PRD를 Gemini CLI / Cursor AI에서 사용할 단계별 프롬프트로 변환해 줘. 5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을 만들어 줘."

단계 분해 결 과 예시

```
# 포트폴리오 PRD 단계별 분해 결과
Step1: HTML 전체 구조 + 섹션 이름 부여 (기능 구현 전)
Step2: performance-metrics 섹션 내용 구체화
Step3: 모던한 CSS 디자인 적용
Step4: 차트·인터랙션 기능 추가
Step5: PRD 기준 검토 + 발견된 개선점 3가지 수정
```

06

Magic Square — Dual-Track UI + Logic TDD 기반 요구사항 작성

UX Contract · Logic Rule · 3단 매핑 표

6교시

60분

"UI는 디자인으로 RED를 만들고, 로직은 규칙으로 RED를 만든다 — GREEN과 REFACTOR는 항상 함께 간다"

UI Track (Boundary 검증)

```
test("입력 전 에러 없음", () => {  
  render(<MagicSquareGrid />);  
  expect(  
    queryByText("오류")  
  ).not.toBeInTheDocument();  
});  
// ✅ UX Contract 기반 RED
```

Logic Track (Control+Entity 검증)

```
def test_invalid_magic_square():  
  ms = MagicSquare([[1,2,3,4],  
    [5,6,7,8],[9,10,11,12],  
    [13,14,15,16]])  
  assert not ms.is_valid()  
  # ✅ Logic Rule 기반 RED
```

⚠️ 두 테스트는 완전히 독립적 — UI 테스트는 로직을 모르고, Logic 테스트는 UI를 모른다

6.2 UX Contract · Logic Rule · 3단 매핑 표 작성

6교시 실습

① UX Contract 언어

- ✓ 보인다 / 안 보인다
- ✓ 가능하다 / 불가능하다
- ✓ 활성화 / 비활성화
- ✓ 포함한다 / 포함하지 않는다

② Logic Rule 언어

- ✓ 허용한다 / 거부한다
- ✓ 유지한다 / 중단한다
- ✓ 반환한다 / 차단한다
- ✓ 계산한다 / 저장한다

③ Magic Square 3단 매핑 표

Scenario	UX Contract	Logic Rule
모든 행 합이 34가 아닌 경우 (L3)	"오류 표시"가 보인다	합 검증이 거부된다
빈칸이 2개가 아닌 경우 (L3)	"빈칸 오류"가 보인다	탐색이 중단된다
올바른 숫자 입력 + 완성 (L1)	"완성!" 메시지가 보인다	해가 반환된다
입력값이 정수 아닌 경우 (L2)	입력 필드가 빨간색으로 표시	입력이 차단된다

⚠ 이 동사들로 끝나지 않는 To-Do는 테스트로 변환하지 말 것 — 판단(Decision)을 포함한 것만 변환 대상

6.3 비교 — 전통적 개발 vs Concept-to-Code Traceability + Dual-Track

비교 항목	전통적 개발	일반 TDD	C2C + Dual-Track TDD
개념 명시성	낮음	낮음	높음 (ECB 스테레오타입)
책임 추적성	없음	약함	강함 (4계층 연결)
실패 조건 정의	늦음 (구현 후)	보통	먼저 (PRD → RED)
UI/Logic 분리	혼재	단일 루프	완전 분리 + Lock-Step
변경 영향 분석	어려움	보통	용이 (Trace 역추적)
도메인 의미 보존	불안정	불안정	안정적 (Drift 감지)
AI 협업 친화성	낮음	보통	높음 (Prompt Chain)
학습 곡선	낮음	보통	처음엔 높음 → 이후 낮음

📌 선택 기준: 개념과 의도를 6개월 후에도 코드에서 읽을 수 있어야 한다면 → C2C Traceability + Dual-Track TDD
"아름다운 소프트웨어란 기능이 동작하는 것이 아니라, 설계 의도가 코드 속에서 살아있는 것이다"

07

Toy Project PRD 작성 + Cursor AI로 개발 시작

AI Assist 기반 소형 프로젝트 시작 · Magic Square PRD 완성

7교시

60분

7.1 PRD 표준 구조 — Magic Square 작성 가이드

7교시

1

Overview (개요)

- 프로젝트 목적 (마방진 완성 지원)
- 문제 정의 (두 빈칸 찾기 + 배치)
- 성공 지표 (테스트 통과율, 성능)

2

Functional Requirements

- Gherkin 기반 Scenario (L0~L3)
- UX Contract (보인다/안 보인다)
- Logic Rule (허용/거부)

3

Non-Functional Requirements

- 성능: 빈칸 탐색 < 100ms
- 보안: 입력 유효성 검증 필수
- 확장성: 5x5 마방진 고려

4

Technical Constraints

- Python 3.10+, pytest, PyQt5
- ECB 3계층 아키텍처
- Git Flow + Cursor AI

5

Out of Scope

- 네트워크 기능 제외
- 다중 사용자 동시 접속 제외
- 5x5 이상 마방진 (Phase 2)



좋은 PRD의 5조건: 명확성 · 측정 가능성 · 달성 가능성 · 관련성 · 완결성 (예외 케이스 포함)

7.2 Gherkin 시나리오 작성 — Magic Square 예시

7교시

Gherkin = Given-When-Then 형식의 시나리오 — 기획자·개발자·QA·Cursor AI 모두가 공유하는 공통 언어

Scenario 1: 정상 마방진 (L1 Happy Path)

Feature: 마방진 검증

Scenario: 올바른 4x4 마방진 검증

Given 모든 행·열·대각선 합이 34인 배열
When `is_valid()` 호출
Then `True` 반환

Scenario 2: 실패 케이스 (L3 Failure Path)

Scenario: 빈칸이 없는 경우 탐색 실패

Given 빈칸이 0개인 완성된 배열
When `find_missing()` 호출
Then `MissingError` 발생

Scenario: 범위 외 숫자 입력

Given 17 이상 숫자 포함 배열
When `validate_range()` 호출
Then `ValidationError` 발생

7.3 Cursor AI로 개발 시작 — AI Assist 기반 워크플로우

7교시 실습

1

PRD → .cursorrules

.cursorrules에 ECB 구조
·테스트 규칙 명시
→ Cursor가 항상 이 규칙 기반 코드 생성

2

Gherkin → RED Test

시나리오를 Cursor Ask
에 붙여넣기
→ "이 시나리오 기반
pytest RED 테스트 생성"

3

RED → GREEN

Cursor Agent: "테스트
를 통과하는
최소 구현 코드를 작성
해줘"

4

GREEN → REFACTOR

Cursor Plan: "코드 품질
개선 계획
→ Agent로 실행"

5

Git Push

git add . && commit →
push
Cursor Chat: "커밋 메시
지 추천해줘"

.cursorrules 핵심 설정

project: MagicSquare · style: PEP8 + 타입힌트 · testing: pytest AAA패턴 · architecture:
ECB(boundary/control/entity) · forbidden: print()디버깅

7.4 Cursor Rules — AI를 Dual-Track TDD 집행자로 만든다

P2

Cursor Rules 철학: "프롬프트 한두 줄이 아니라, AI를 TDD 실행 엔진으로 고정하기 위한 강제 규칙 세트"

Cursor Rules = Concept-to-Code Traceability를 AI 협업에서 자동화하는 메커니즘

RED 선언 원칙

코드 작성 전 반드시 실패 테스트 먼저
UX Contract 또는 Logic Rule 기반으로만

ECB 무결성

Entity: 비즈니스 데이터만
Control: 조율과 규칙만
Boundary: 인터페이스만

Traceability 주석

모든 클래스에 ECB 스테레오 타입 명시
Concept Trace + Responsibility 포함

Lock-Step REFACTOR

모든 테스트 GREEN 확인 후 리팩토링
UX 행동 변화 금지

실전 Cursor 프롬프트 패턴

RED 시작

We start with UI/UX RED.
Write failing UI tests based on Figma behavior.
Do NOT implement yet.

GREEN 연결

Logic GREEN first.
Implement domain logic only to satisfy logic tests.
No UI changes.

REFACTOR

All tests pass.
Refactor UI and logic together.
Do not change UX behavior.
Maintain ECB boundaries.

.mdc 例

1. 프로젝트 전용 룰 (Project Rules) - 권장

특정 프로젝트에만 적용되는 룰입니다. 프로젝트의 최상위 루트 디렉토리에 파일로 생성합니다.

파일 위치: 프로젝트 폴더 가장 최상위(.cursor/rules)

파일명: .cursorrules (또는 .cursor/rules/*.mdc)

설정 방법: 프로젝트 폴더 내에 .cursorrules 파일을 만들고, AI가 지켜야 할 코드 스타일, 네이밍 규칙, 기술 스택 등을 텍스트로 작성하면 됩니다.

2. 글로벌 룰 (User Rules) - 공통

모든 프로젝트에 공통으로 적용되는 룰입니다.

위치: 커서 설정 화면 (Cursor Settings > General > Rules for AI)

설정 방법:

커서 실행

우측 상단 설정 버튼 클릭

General 탭 내 Rules for AI 입력창에 규칙 작성

7.5 Cursor AI로 개발 시작 — AI Assist 기반 워크플로우

7교시 실습



Cursor Rules 고급 설정 (.cursorrules)

.cursorrules - 코드 리뷰 & 기능 생성 규칙

```
# === 코드 리뷰 요청 시 ===
When asked to review code, always check:
1. SOLID 원칙 위반 여부 (각 항목 설명 포함)
2. 보안 취약점 (OWASP Top 10 기준)
3. 성능 개선 포인트 (Big-O 포함)
4. 테스트 커버리지 제안
5. 마크다운 표 형식으로 정리

# === 새 기능 구현 요청 시 ===
When implementing features, always:
1. 기존 패턴 (@src/services 참고)을 따를 것
2. TypeScript strict mode 준수
3. 구현 후 테스트 코드도 함께 생성
```

.cursorrules - 자동화 파이프라인 규칙

```
# === PR 준비 체크리스트 ===
Before finalizing any code, verify:
□ npm run build 통과
□ npm test 통과 (커버리지 80%+)
□ ESLint 에러 없음
□ 모든 함수에 JSDoc 주석
□ README.md 관련 섹션 업데이트

# === 파일 생성 컨벤션 ===
- Service 파일: src/services/*.service.ts
- Route 파일:  src/routes/*.route.ts
- Type 파일:   src/types/*.type.ts
- Test 파일:   src/__tests__/*.test.ts
```



Chat 자동화 & 반복 작업 처리

Chat 반복 작업 자동화 예시

```
# 1. 테스트 없는 파일 일괄 처리
"@Codebase 테스트 파일이 없는 서비스를
찾아서 모두 Jest 테스트 작성해줘"

# 2. JSDoc 일괄 추가
"@src/services 폴더의 모든 public
메서드에 JSDoc 주석 추가해줘"

# 3. 의존성 최신화
"package.json 분석하고 보안 취약점 있는
패키지 최신 버전으로 업데이트해줘"
```

GitHub Actions + Cursor 규칙 연동

```
# .github/workflows/ai-check.yml
- name: Lint Rules Validation
  run: |
    # .cursorrules에 정의한 규칙을 CI에서도 검사
    npm run lint
    npm run typecheck
    npm test -- --coverage --
    coverageThreshold='{ "global": { "lines": 80 } }'
# PR 머지 전 자동 품질 게이트
```

7.5 Cursor AI로 개발 시작 — AI Assist 기반 워크플로우

7교시 실습

.mdc 例

프로젝트\.cursor\rules\.mdc

▼ MAGICSQUARE_1004

▼ .cursor\rules

- ↓ magicsquare-ecb-architecture.mdc
- ↓ magicsquare-forbidden.mdc
- ↓ magicsquare-project.mdc
- ↓ magicsquare-python-code-style.mdc
- ↓ magicsquare-tdd-testing.mdc

▼ MAGICSQUARE_1004

> .cursor\rules

> Prompting

> Report

! .cursorrules

7.6 Cursor AI 에이전트 체인 — 자율 실행 파이프라인

7교시 실습



Cursor Composer + Chat = AI 에이전트 루프 (Thought → Action → Observation 반복)


사용자 목표
입력


Cursor
분석 (Thought)


Action
(파일 수정·실행)


Observation
(결과 확인)


완료 또는
반복

반복 (목표 달성까지)



실전 에이전트 — PR 자동 리뷰 (Cursor Chat + OpenRouter)

Cursor Chat에서 PR 리뷰 에이전트 실행

1. Chat에서 @terminal과 @git 컨텍스트 활용

"@terminal 'git diff HEAD~1'을 실행해서 변경 파일 목록을 가져오고,
각 파일을 @코드베이스에서 읽어 다음을 분석해줘:

1) 보안 취약점 (OWASP Top 10), 2) 테스트 누락, 3) 성능 이슈

결과를 review_report.md에 저장하고, 치명적 이슈가 있으면 'BLOCK', 없으면 'APPROVE'로 마무리해줘."

Composer가 자율적으로: git diff 실행 → 파일 읽기 → 분석 → 리포트 작성 → 결론 도출



Cursor의 Composer는 @terminal을 통해 실제 명령어를 실행하고 결과를 반영합니다. 에이전트처럼 반복 루프를 돌며 목표를 달성합니다.

7.7 OpenRouter API — 멀티모델 라우팅

7교시 실습

🌐 OpenRouter란? — 단일 API로 100+ LLM 모델에 접근 / Cursor AI와 함께 사용

개발자 앱
(단일 API 키)



OpenRouter
라우터

스마트 라우팅 예시 (JS)

```
const client = new OpenAI({ baseUrl:"https://openrouter.ai/api/v1",  
  apiKey:process.env.OPENROUTER_API_KEY });  
async function smartChat(prompt, complexity="medium") {  
  const modelMap = { simple:"google/gemini-flash-1.5",  
    medium:"anthropic/claude-3.5-sonnet", complex:"anthropic/claude-  
    opus-4" };  
  return client.chat.completions.create({  
    model:modelMap[complexity], messages:[{role:"user",content:prompt}]  
  });  
}
```

🖱️ Cursor AI에서 OpenRouter 모델 사용하기

Cursor Settings > Models > Add Custom Model

```
baseUrl: https://openrouter.ai/api/v1  
apiKey: OPENROUTER_API_KEY  
model: anthropic/claude-3.5-sonnet (또는 원하는 모델)
```

Claude 3.5
Sonnet

GPT-4o

Gemini 1.5
Pro

Llama 3.1
(오픈소스)



비용 최적화

태스크별 최적 모델
선택
(빠른 작업은 저렴한
모델)



Fallback 처리

모델 장애 시 자동 전
환
99.9% 가용성 보장



사용량 추적

모델별 비용·토큰 사
용량
대시보드에서 통합 관
리



오픈소스 접근

Llama, Mistral 등
무료 모델도 동일 API

OpenRouter API 연동 — 무료 AI 모델 활용

🗨️ 텍스트 모델

모델	상태	비고
meta-llama/llama-3.3-70b-instruct:free	❌ Rate-limited	일시적, 나중에 재시도 가능
meta-llama/llama-3.2-3b-instruct:free	❌ Rate-limited	일시적
qwen/qwen3-coder:free	❌ Rate-limited	일시적
nvidia/nemotron-3-super-120b-a12b:free	✅ 정상	응답 빠름 (3.5s)
nvidia/nemotron-3-nano-30b-a3b:free	⚠️ 빈 응답	모델 자체 문제

🖼️ 비전(이미지) 모델

모델	상태	비고
google/gemma-3-27b-it:free	❌ Rate-limited	일시적
google/gemma-4-31b-it:free	❌ Rate-limited	일시적
google/gemma-4-26b-a4b-it:free	✅ 정상	강마지 이미지 정확히 인식 (4.1s)
nvidia/nemotron-nano-12b-v2-vl:free	⚠️ 빈 응답	모델 자체 문제

7.8 MCP 입문 — AI를 실행자로 만드는 도구

7교시 실습

Model Context Protocol (MCP)

2024년 11월 Anthropic이 발표한 오픈 표준 — AI 모델(LLM)과 외부 데이터 소스·도구 사이의 연결을 표준화하는 개방형 프로토콜

💡 비유: AI를 위한 USB-C 포트



USB-C

어떤 기기도 같은 포트로 연결



MCP

어떤 AI도 같은 방법으로 외부 연결

📌 핵심 사실



2024.11.25 발표

Anthropic이 오픈소스로 공개



오픈 표준

OpenAI·Google DeepMind도 공식 채택



N×M 문제 해결

기존 커스텀 커넥터 방식 대체



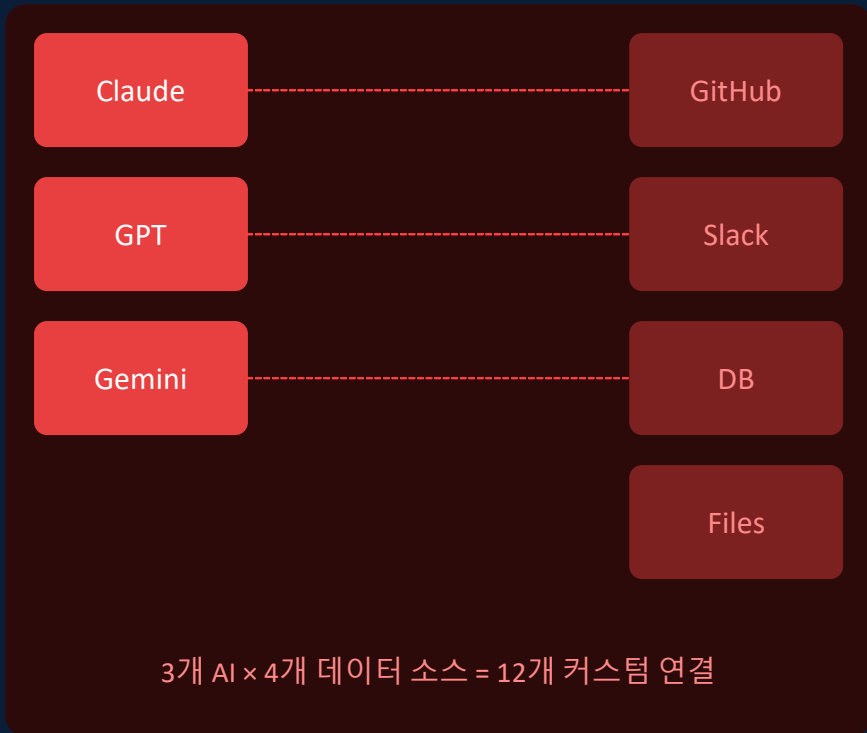
Linux Foundation 이관

2025.12 AAIF로 중립성 확보

7.8 MCP 입문 — AI를 실행자로 만드는 도구

7교시 실습

✗ MCP 이전: $N \times M$ 통합 지옥



✓ MCP 이후: 표준 1개로 모두 연결



7.8 MCP 입문 — AI를 실행자로 만드는 도구

7교시 실습



MCP Host

(예: Claude Desktop)

사용자가 직접 사용하는 AI 애플리케이션
여러 MCP Client를 포함·관리

예시

Claude Desktop, Cursor IDE, VS Code

↕ JSON-RPC 통신



MCP Client

(프로토콜 처리)

Host 내부의 프로토콜 처리 담당
MCP Server와 1:1 연결 설정

예시

Host 내 내장 컴포넌트

↕ JSON-RPC 통신



MCP Server

(기능 제공)

특정 도구·데이터 소스 기능 제공
Resources, Tools, Prompts 노출

예시

Filesystem, GitHub, Slack, DB 서버

MCP 아키텍처 — Host · Client · Server

데이터 흐름: 사용자 요청 → Host → Client → Server → 외부 데이터/도구 → Client → Host → 사용자 응답

7.8 MCP 입문 — AI를 실행자로 만드는 도구

7교시 실습

MCP 서버 저장소 = 개발자들이 만든 MCP 서버를 공유·검색·설치할 수 있는 중앙 허브

전통적인 npm·pip 패키지 저장소처럼, AI가 사용할 수 있는 도구(Tools)·데이터(Resources)를 모아놓은 공간입니다.



GitHub (공식 서버)

github.com/modelcontextprotocol/servers

Anthropic이 직접 관리하는 공식 레퍼런스 서버 모음
파일시스템·Git·Brave Search·PostgreSQL 등 포함



Smithery

smithery.ai

5,900개+ MCP 서버 등록된 세계 최대 MCP 마켓플레이스
클릭 몇 번으로 설치·연결 가능, GitHub 계정으로 가입

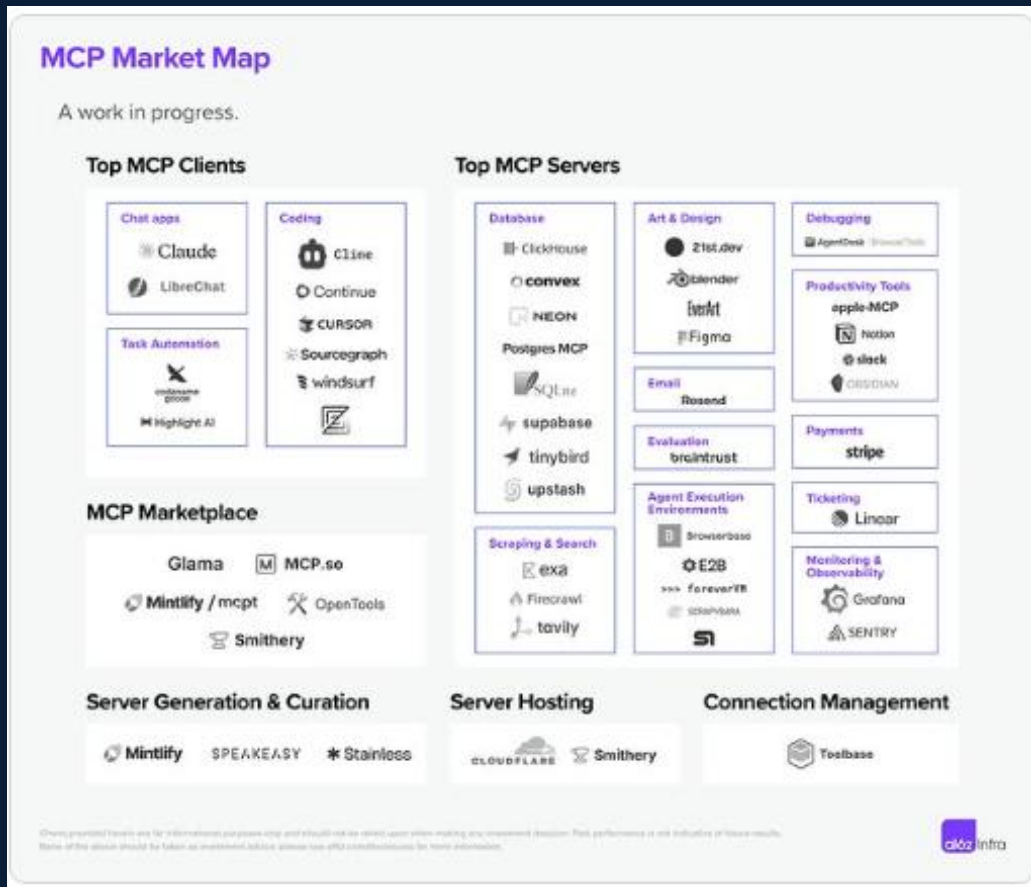


awesome-mcp-servers

github.com/punkpeye/awesome-mcp-servers

커뮤니티가 엄선한 고품질 MCP 서버 큐레이션 목록
카테고리별 분류 (개발·검색·업무·데이터 등)

사전 구축된 MCP 서버 생태계



7.8 MCP 입문 — AI를 실행자로 만드는 도구

7교시 실습

Smithery = MCP 서버의 앱스토어 + 중앙 레지스트리 + 개발 플랫폼

5,900개+ 서버 보유 · Claude Desktop·Cursor·VS Code 등 모든 MCP 클라이언트 지원 · GitHub 계정으로 가입



서버 탐색

카테고리·검색으로
원하는 MCP 서버 탐색



원클릭 설치

JSON 복사 또는 CLI
명령어로 즉시 설치



Playground

설치 전 서버 기능을
온라인에서 미리 테스트



서버 배포

내가 만든 서버를
Smithery에 공개 배포



API Key 관리

서버별 인증 키 발급
안전하게 관리



Observability

서버 사용량·오류
모니터링 대시보드

클라이언트 무관(client-agnostic) 설계 — 어떤 MCP 클라이언트에서도 Smithery 서버 사용 가능

7.8 MCP 입문 — AI를 실행자로 만드는 도구

7교시 실습

Sequential Thinking이란?

복잡한 문제를 단계적 사고(step-by-step reasoning)로 분해하여 Claude의 추론 품질을 높이는 MCP 서버
→ 수학·코드 설계·전략 계획 등 복잡한 작업에서 오류를 줄이고 체계적 사고를 강제



수학·계산

복잡한 수식을 단계별로 분해
중간 계산 포함 정확도 향상



시스템 설계

아키텍처 결정을 순서대로
각 단계 근거와 함께 제시



디버깅

버그 원인 추적을
논리적 순서로 분석



전략 계획

복잡한 프로젝트 계획을
의존성 순서로 정리

7.8 MCP 입문 — AI를 실행자로 만드는 도구

7교시 실습

Context7이란?

프로그래밍 라이브러리의 최신 공식 문서와 코드 예제를 실시간으로 AI에 제공하는 MCP 서버
→ Claude가 최신 React·FastMCP·Next.js 등의 정확한 API를 코딩 시 직접 참조

✗ Context7 없이

- Claude가 학습 당시의 오래된 버전 API 사용
- 존재하지 않는 메서드 Hallucination
- 버전 충돌 오류 코드 생성
- "이 코드 안 돼요" 반복 디버깅

✓ Context7 사용 시

- 프롬프트에 "use context7" 추가
- 최신 공식 문서 기반 코드 생성
- 실제 동작하는 정확한 예제 제공
- 라이브러리 변경사항 즉시 반영

1일차 강의 자료 · 8시간

워크 북 실습

실습 #- .cursorrules 작성

실습 # - API 활용 & AI Agent

실습 # - Cursor.AI 로 MCP 사용하기

08

To-Do 구조 작성 원리

체크박스 · 단계별 번호 · Requirements 추적 · 체크포인트 · 속성 테스트 · 검증 포인트

8교시

60분

핵심 전제: "모든 to-do는 테스트로 변환되지 않는다. 판단(Decision)을 포함한 to-do만 테스트로 변환된다"

구분	To-Do List	Work Order
목적	해야 할 일 목록화	LLM에게 실행 지시
수준	개략적, 의도 중심	구체적, 계약 중심
예시 내용	"마방진 검증 구현"	def is_valid(grid:List[List[int]])->bool: 반환, pytest AAA패턴
사용 시점	기획·분석 단계	개발·구현 단계
효과	우선순위 정리	AI 할루시네이션 방지

Work Order 예시 (Magic Square)

WI-001: SquareValidator.is_valid(grid) 구현 · ECB: <<control>> · 입력: 4x4 List[List[int]] · 반환: bool
실패조건: 합#34 → False · 테스트: pytest AAA · 검증: test_invalid_magic_square() PASS

① 나쁜 프롬프트 vs ② 좋은 프롬프트 — C2C Traceability 기반 Work Order 생성

✖ 나쁜 프롬프트

"Magic Square TDD로 만들어줘"

문제점:

- 테스트 통과 기준 없음
- ECB 구조 미지정
- 어떤 to-do가 테스트인지 불명
- 검증 방법 없음
- Hallucination 위험 높음
- 범위 불명확 → 과도한 구현

✔ 좋은 프롬프트 (Work Order 구조)

Context

TDD RED→GREEN 전환 단계

ECB: MagicSquare<<entity>>

Test (첨부: @test_magic.py)

```
def test_invalid_square(): ...
```

제약 조건

1. 테스트 통과 최소 코드만
2. 타입힌트 필수
3. 하드코딩 허용 (GREEN)
4. SOLID 불필요 (REFACTOR)

요청: magic_square.py 작성

8.3 사용자 여정(User Journey) → To-Do 구조화

8교시

💡 핵심 전제: "To-Do는 사용자가 시스템과 상호작용하는 순간에서 나온다 — 행위가 없으면 To-Do도 없다"

🌐 사용자 여정이란?

진입점 (Entry)

- 1 사용자가 시스템에 접근하는 첫 행위
예: 4x4 Magic Square 게임 시작



핵심 행위 (Core Action)

- 2 목적을 달성하기 위한 주요 상호작용
예: 숫자 입력 → 유효성 검사 요청



결과 확인 (Outcome)

- 3 시스템이 반환하는 피드백 수신
예: 합계 34 성공 / 오류 메시지

✏ 여정 → To-Do 변환 규칙

사용자 여정 문장

→ To-Do 항목

사용자가 __ 한다

→ [] __을 구현한다

시스템이 __을 보여준다

→ [] __ UI 컴포넌트 생성

__이 실패하면

→ [] 실패 케이스 테스트 작성

__을 확인한다

→ [] 검증 로직 + AAA 테스트

◆ MagicSquare 여정 → To-Do 예시

사용자가 4x4 격자에 숫자를 입력한다

→ [] GridUI <<boundary>> 4x4 테이블 구현

→ [] SquareValidator.is_valid(grid) 구현

8.4 PRD 작성 → Work Order 자동 생성 구조

8교시

💡 핵심 전제: "PRD는 판단(Decision)을 포함한 To-Do만 Work Order로 변환하는 필터다"

PRD 5요소 → To-Do 연결 구조 (MagicSquare 기준)

① 목표 (Goal)	② 사용자 (Who)	③ 기능 (What)	④ 제약 (Constraint)	⑤ 검증 (Acceptance)
4x4 Magic Square 유효성 검사 및 빈칸 탐색 구현	퍼즐 게임 플레이어 — 4x4 격자 숫자 입력	합계 검증 / 빈칸 탐색 / 결 과 표시	ECB 레이어 준수 pytest 커버리지 80%+	합≠34 → False 빈칸=0 → raise ValueError
→ To-Do [] SquareValidator 모듈 구현	→ To-Do [] GridUI <<boundary>> 설계	→ To-Do [] is_valid() find_missing() display()	→ To-Do [] .cursorrules 적용 + 테스트 선행 작성	→ To-Do [] AAA 패턴 테스트 케이스 명세

⚙️ To-Do → Work Order 변환 필터 (판단 포함 여부 기준)



✅ ④제약 + ⑤검증에 판단이 있어야 Work Order가 생성된다

Work Order 예시 (MagicSquare)

```
WI-002: is_valid(grid)->bool, 합≠34→False · ECB:  
<<control>>  
실패조건: 합≠34→False · 테스트: pytest AAA ·  
PASS
```

1일차 강의 자료 · 8시간

워크 북 실습

실습- 사용자 여정 작성

실습- 최종 PRD 작성 (Gherkin 포함)

09

To-Do 리스트 생성 요청 프롬프트 실습

Work Order 구조 · Cursor AI 협업 전략 · TDD 착수 점검

1교시

60분

9.1 To-Do → Work Order — Cursor AI 협업의 핵심 구조

8교시

핵심 전제: "모든 to-do는 테스트로 변환되지 않는다 — 판단(Decision)을 포함한 to-do만 변환된다"

To-Do List

"무엇을 할 것인가"

[] Magic Square Entity 정의

[] SquareValidator 구현

[] 빈칸 탐색 알고리즘

[] UI 격자 표시

[] 결과 출력

Work Order

"어떻게, 검증은"

WI-001: MagicSquare(size) → ECB <<entity>>

WI-002: is_valid(grid)→bool, 합#34→False

WI-003: find_missing()→list, 빈칸0개→raise

WI-004: GridUI <<boundary>>, 4×4 테이블

WI-005: ResultDisplay, 해/실패 메시지

To-Do : Work Order = 1 : N · 하나의 할 일 → 여러 세부 작업 지시 · Work Order = 검증 가능한 계약

9.2 To-Do → Test 변환 핵심 규칙

8교시

Rule 0

전제 선언

판단 포함 to-do만 변환

Rule 1

행동형 폐기·분해

"로그인 화면 만들기" → "언제 거부?"로

Rule 2

상태 조건 필수

"언제(When)" 없으면 테스트 불가

Rule 3

Scenario 매핑

L0~L3 중 하나에 속해야 함

Rule 4

UX/Logic 분리

"에러 보임(UX)" + "제출 거부(Logic)"

Rule 5

언어 제한

보인다/허용한다 → 변환 가능

Rule 6

1:1 대응

to-do 1개 = 테스트 1개만

Rule 7

이름 = 문장

테스트명 = to-do 문장 그대로

Rule 8

실패 의미 선언

실패 = 개념·책임 위반 가능성

Rule 9

생명주기 일치

추가→RED, 완료→GREEN, 삭제→테스트삭제

📄 체크박스 & 단계별 번호 체계

Epic-001: 마방진 검증 시스템

US-001: 합 검증

- [x] TASK-001: MagicSquare Entity 정의
- [] TASK-002: SquareValidator 구현
 - [] TASK-002-1: is_valid() RED
 - [] TASK-002-2: is_valid() GREEN
 - [] TASK-002-3: REFACTOR

US-002: 빈칸 탐색

- [] TASK-003: MissingFinder 구현
 - * 선택: N-Queen 알고리즘 활용
 - > [Checkpoint] 성능 < 100ms 검증

🔗 Requirements 추적 매트릭스

Task ID	Req ID	Scenario	테스트	상태
TASK-001	REQ-001	L1 Happy	test_valid_ms	✅ PASS
TASK-002	REQ-002	L3 Fail	test_invalid	❌ RED
TASK-003	REQ-003	L2 Edge	test_missing	🟡 TODO
TASK-004	REQ-001	L1 UI	test_grid_ui	🟡 TODO

추적성: Task → Req → Scenario → Test
이 연결고리가 C2C Traceability의 실천

✓ 체크박스 항목

- [] 미완료
 - [x] 완료
 - [-] 불필요(취소)
- 완료 시 연결 테스트 GREEN으로 자동 전환

12 단계별 번호

Epic → User Story → Task → Sub-Task
계층 구조로 의존성 명확화
Epic-001 > US-001 > TASK-001-1

* 선택 항목 (*)

- * 선택: 알고리즘 최적화
비필수 기능을 명확히 구분
스프린트 범위 관리에 필수

> 체크포인트

- > [Checkpoint] 성능 < 100ms
 - > [Checkpoint] 커버리지 80%+
- 특정 기준 달성 시 다음 단계 진행

🔧 속성 테스트

```
@given(valid_grid)
def test_valid_range(grid):
    assert all(1<=v<=16 for row ...)
랜덤 입력으로 속성 불변성 검증
```

✓ 검증 포인트

[Validate] is_valid() → True
[Validate] 커버리지 80%+
[Validate] 성능 < 100ms
각 Task 완료 기준 명시

9.5 Magic Square — 완성 To-Do 구조 예시

8교시 실습

이 구조가 To-Do → Scenario → Test → Code의 C2C Traceability 완전 구현 예시입니다

Epic-001: Magic Square 4x4 완성 시스템

US-001: 마방진 유효성 검증 (REQ-001)

- [x] TASK-001: MagicSquare Entity 정의 (<<entity>>) → test_entity_creation PASS
- [] TASK-002: SquareValidator.is_valid() 구현 (<<control>>)
 - [] TASK-002-1: [RED] test_invalid_magic_square() 작성
 - [] TASK-002-2: [GREEN] 최소 구현으로 통과
 - [] TASK-002-3: [REFACTOR] 타입힌트 + 독스트링
- > [Checkpoint] test_invalid_magic_square 통과 확인 후 진행

US-002: 빈칸 위치 탐색 (REQ-002)

- [] TASK-003: MissingFinder.find_missing() 구현 (<<control>>)
 - [] TASK-003-1: [RED] test_missing_not_found() 작성 (L3 Failure)
 - [] TASK-003-2: [GREEN] 빈칸 탐색 최소 구현
- * 선택: Backtracking 최적화 알고리즘
- > [Checkpoint] 성능 < 100ms 검증 · [Validate] 커버리지 80%+

2일차 강의 자료 · 8시간

워크 북 실습

실습- Cursor AI To-Do 리스트와 README.md 파일 작성

여기까지 작업한 다음 깃헙 저장소를 만들고 깃헙에 업로드합니다.

1일차 핵심 정리

01

Prompt Engineering

P-C-T-F(기본) + 7단계(복잡한 설계) + Few-Shot/CoT/ReAct(고급)

Cursor AI 3모드: Agent(실행) · Plan(계획) · Ask(이해) · .cursorrules로 일관성 확보

02

C2C Traceability + 의사결정 파이프라인

문제인식 → 온톨로지(ECB) → 프롬프팅 체인 = 의미 보존의 실천 공식

4계층: Concept → Responsibility → Test → Code — 설계 의도가 코드까지 살아있어야 한다

03

Magic Square × ECB × 문제인식

Entity(MagicSquare) · Control(SquareValidator) · Boundary(GridUI)

L3 실패 조건부터 시작: "언제 거부하는가"가 진짜 요구사항

04

Dual-Track TDD + Scenario + PRD

왜 늦게 실패? → PRD = 테스트의 원천 → Gherkin L0~L3 시나리오 → UX/Logic 분리

To-Do 구조: 체크박스 · 번호 · 추적 · 체크포인트 · 속성테스트 · 검증포인트

내일 2일차: TDD RED → GREEN → REFACTOR 완전 구현 · Cursor Agent 실전 활용 · Prompting Best Practices

THANK YOU

앞으로의 엔지니어는
단순한 '코더'가 아니라
AI와 협력하여 문제를 해결하는
지식 설계자입니다.